

10

فريق كتابة المقررات الجامعية

بنيان الحاسب - Computer architecture

1 مقرر الدكتور مازن

السلام عليكم يا أصدقاء

نقدم لكم الجزء الثاني من مادة بنيان الحاسب للدكتور مازن محايري

نتمنى لكم دوام التوفيق 😊

المحاضرات	رقم الصفحة
المحاضرة الأولى	1
المحاضرة الثانية	8
المحاضرة الثالثة	17
المحاضرة الرابعة	28
المحاضرة الخامسة	38

المحاضرة الأولى

البنيان Architecture : هي مواصفات النظام الحاسوبي الظاهرة بالنسبة للمبرمج أو التي تؤشر على التنفيذ المنطقي للبرنامج

يجب على المبرمج معرفة ما يلي لكتابة برنامج بلغة الآلة :

- التعليمات
- عدد البتات التي تستخدم لتمثيل مختلف أنواع المعطيات (المحارف والأرقام)
- طرق العنونة (أضع عنوان وأحصل على قيمة)
- آليات الإدخال والإخراج

التنظيم Organization : هي الوحدات المشكلة للنظام الحاسوبي cpu, memory, I\O unit

والتوصيلات بينها والتي تحقق المواصفات البنيوية أي تشمل تفاصيل العتاديات (HW) الشفافة بالنسبة للمبرمج

مثال : إشارات التحكم التي تصدرها cpu لتنفيذ التعليمة، تقانة الذاكرة المستخدمة، واجهات الربط بين الوحدات (الأجهزة) والحاسوب (InterFace)

فرضا : يوجد تعليمة ضرب هي قضية تصميم بنيوي أما تنفيذ هذه التعليمة وإنجازها Implementetion فهي قضية تنظيمية حيث تنفذ إما بضارب خاص أو باستخدام دارة الجمع استخداما مكررا

سندرس مكونات الحاسوب pc الأساسية :

- Memory بكل أشكالها : أ- الذاكرة الرئيسية main memory

  ب- ذاكرة الكاش cache

  ج- الذاكرة الافتراضية virtual
- طرق العنونة Addressing mode
- I/O unit
- خطوط اتصالات تربط العناصر ببعضها وهي bus بربط بين الوحدات المختلفة
- تصميم cpu

Memory (semiconductor)	
ROM Read Only Memory لـ تفقد محتواها عند انقطاع التغذية	RAM Randome Access Memory Read Write Memory (RWM) تقفذ محتواها عند انقطاع التغذية

الذاكرة عبارة عن مجموعة من الكلمات

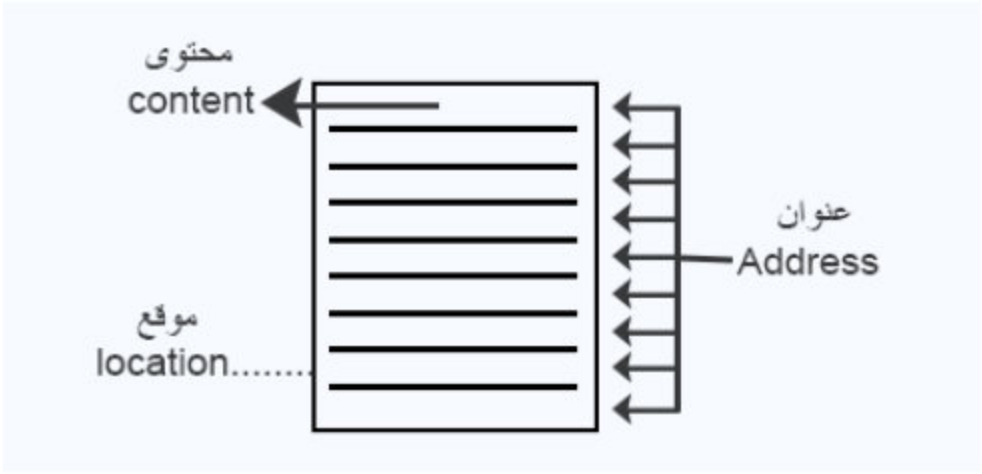
ال location يحوي بداخله على content أو data ولكل location عنوان

تختلف الكلمات (1 بت ، 8 بت ....)

تصنيف الذواكر حسب طرق الوصول للمعطيات المخزنة فيها :

- RAM , ROM : ذاكرات الوصول العشوائي أي أن زمن الوصول للموقع لا يتعلق بمكان الموقع

مثال : Rom, Ram, SSD

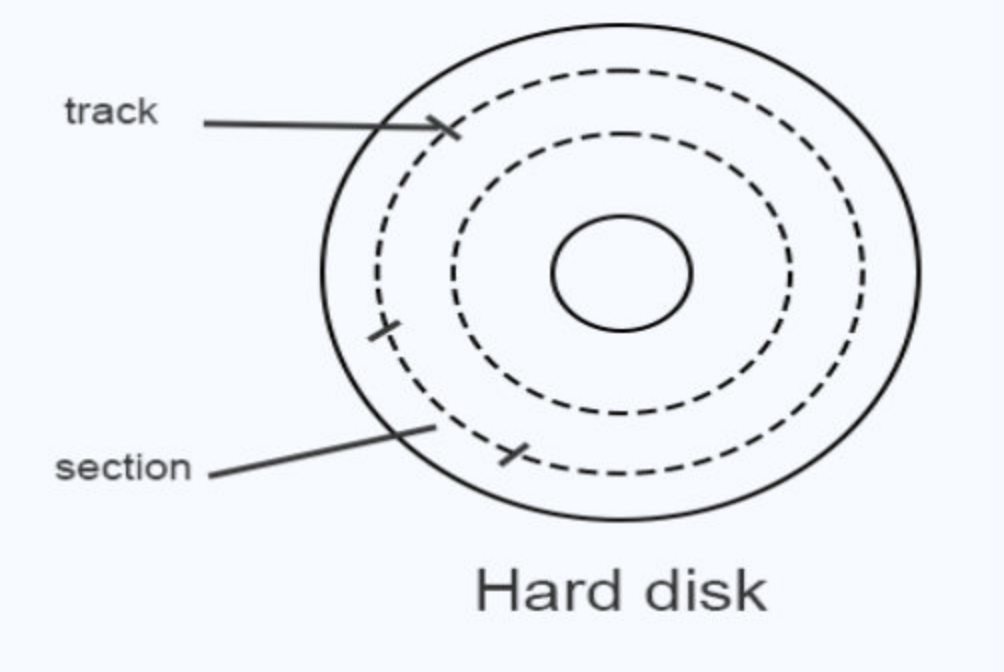


- SAM : Sequential Access Memory : ذاكرات الوصول التسلسلي - مثال : (tape) شريط

زمن الوصول Access time للموقع يتعلق بمكانه

زمن الوصول : هو الزمن الفاصل بين وضع العنوان وقراءة أو تخزين data

- Direct (المباشر) : الوصول إلى المسار (track) ثم إلى القطاع (section)



- Associative التجميعي : الوصول حسب المحتوى وليس العنوان (مثال : cache) يستخدم مقارنات للوصول للمحتوى

	أنواع RAM	
	Dynamic DRAM	Static SRAM
الاستخدام الرئيسي	ذاكرة رئيسية main memory	cache
خلية التخزين cell	مكثفة Mos (ترانزستور واحد) (Tr-1)	(Flip Flop) Latch (Tr 4-6)
المحاسن والمساوئ	مساحة أقل (حجم) كثافة تخزين عالية تكلفة أقل استهلاك أقل للطاقة أبطأ بحاجة لإنعاش دوري refresh	مساحة أكبر كثافة تخزين صغيرة تكلفة أعلى استهلاك أعلى للطاقة أسرع ليس بحاجة لإنعاش
أنواع	SDRAM (sync) DDR (double data raten)	
	Mos إجباري	تصمم من Bjt, Mos

في اللابتوبات عند ذكر ما يسمى مثلا 8GB RAM عندها يقصد بها : DRAM

أنواع ROM	
التصنيع	
Mos, BJT	MROM (mask) مبرمجة لمرة واحدة من المعمل وتستخدم عند الحاجة لكميات كبيرة
Mos, BJT	PROM (programmable) وتسمى أيضا (OTP) one time programmable مبرمجة لمرة واحدة في مكان العمل (الحقل) وليس بالمعمل
Mos	EPROM (Erasable) قابل للمحي الضوئي (بالأشعة فوق البنفسجية)
Mos	EEPROM (Electrically) الحذف الكهربائي
Mos	Flash (EEPROM نوع من) الحذف الكهربائي والكتابة السريعة الاستخدام الرئيسي : 1) ذاكرة إقلاع الحاسوب BIOS والتي تحوي برنامج إقلاع الحاسوب 2) التجهيزات الرئيسية Basic I\O system

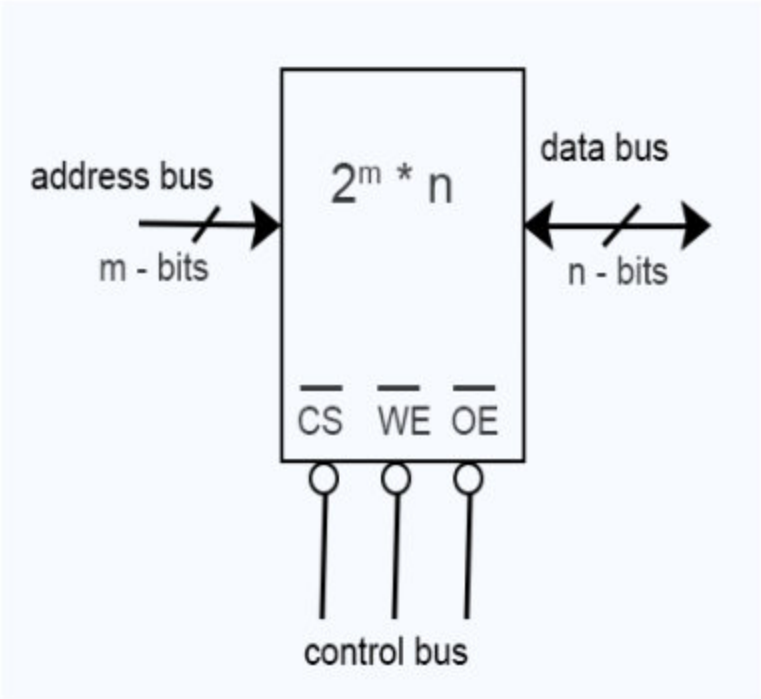
الـ PROM و EPROM لا تكون البرمجة والمحي بالدارة مباشرة بل تزال ثم تعاد

طورت هاتين الذاكرتين إلى EEPROM

ملاحظة:

أسرع الذاكرات هي SRAM ثم DRAM ثم ROM

SRAM (ليست ك cach) : السؤال يأتي مثلا : ارسم البنية الداخلية للذاكرة:



الادخال والإخراج ليس في آن واحد

عدد الكلمات : 2^m

n : هي عرض الكلمة width of word

(عدد البتات التي بتم التعامل معها بنفس الوقت في عملية واحدة : كتابة أو قراءة)

CS : chip select

OE : output Emable

WE : write Emable

الوحدة الأساسية ل data هي 8bits أو من مضاعفاتها

حفظ

1M == > 2^{20}

1K == > 2^{10}

1G == > 2^{30}

1T == > 2^{40}

مثال : 4GB

4GB <== > 4G * 8

n = 8 bits

m =?

$2^m = 4G = 2^2\ 2^{30} = 2^{32}$

m = 32 bits

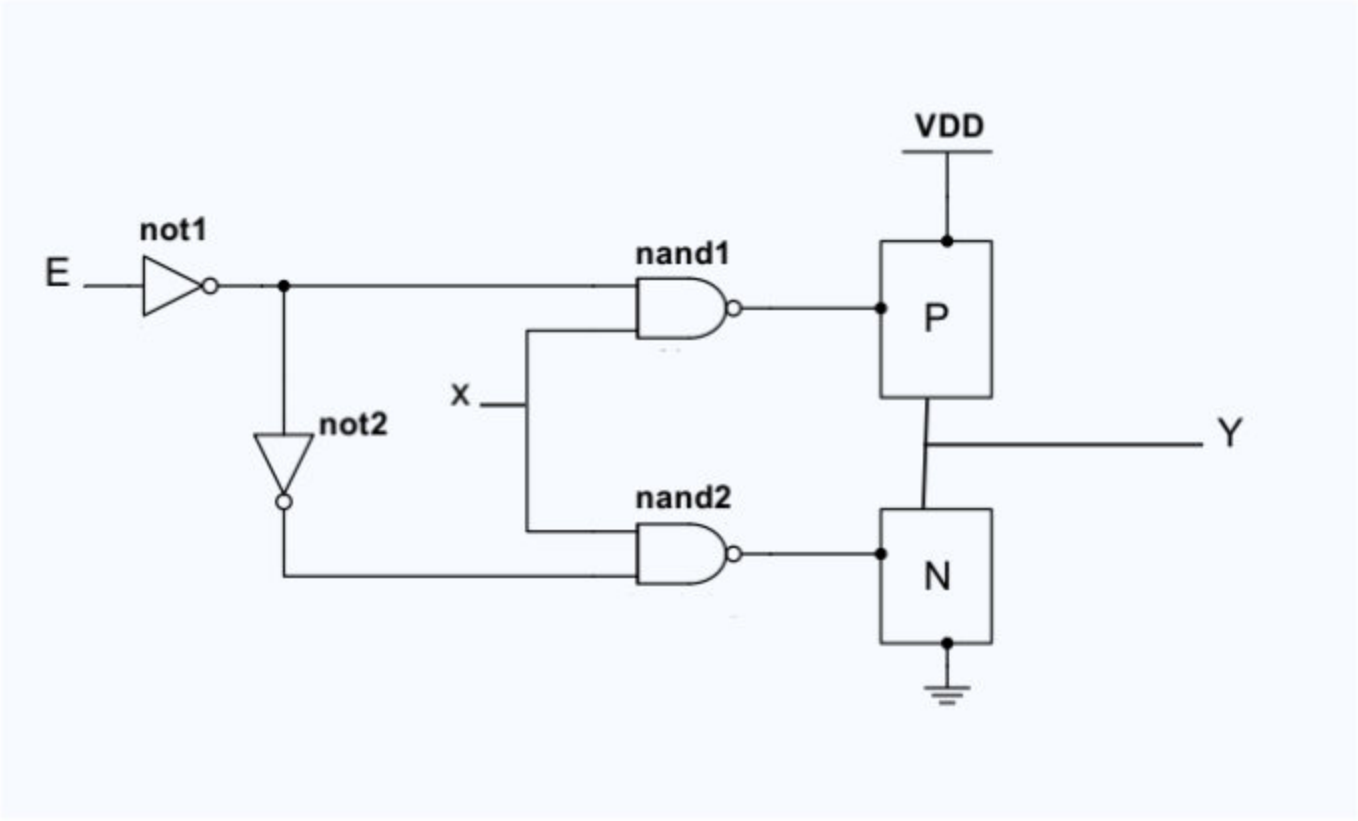
مثال : 32M * 16

n = 16 bits

m =?

$$32M = 2^5\ 2^{20} = 2^{25}$$

m = 25 bits



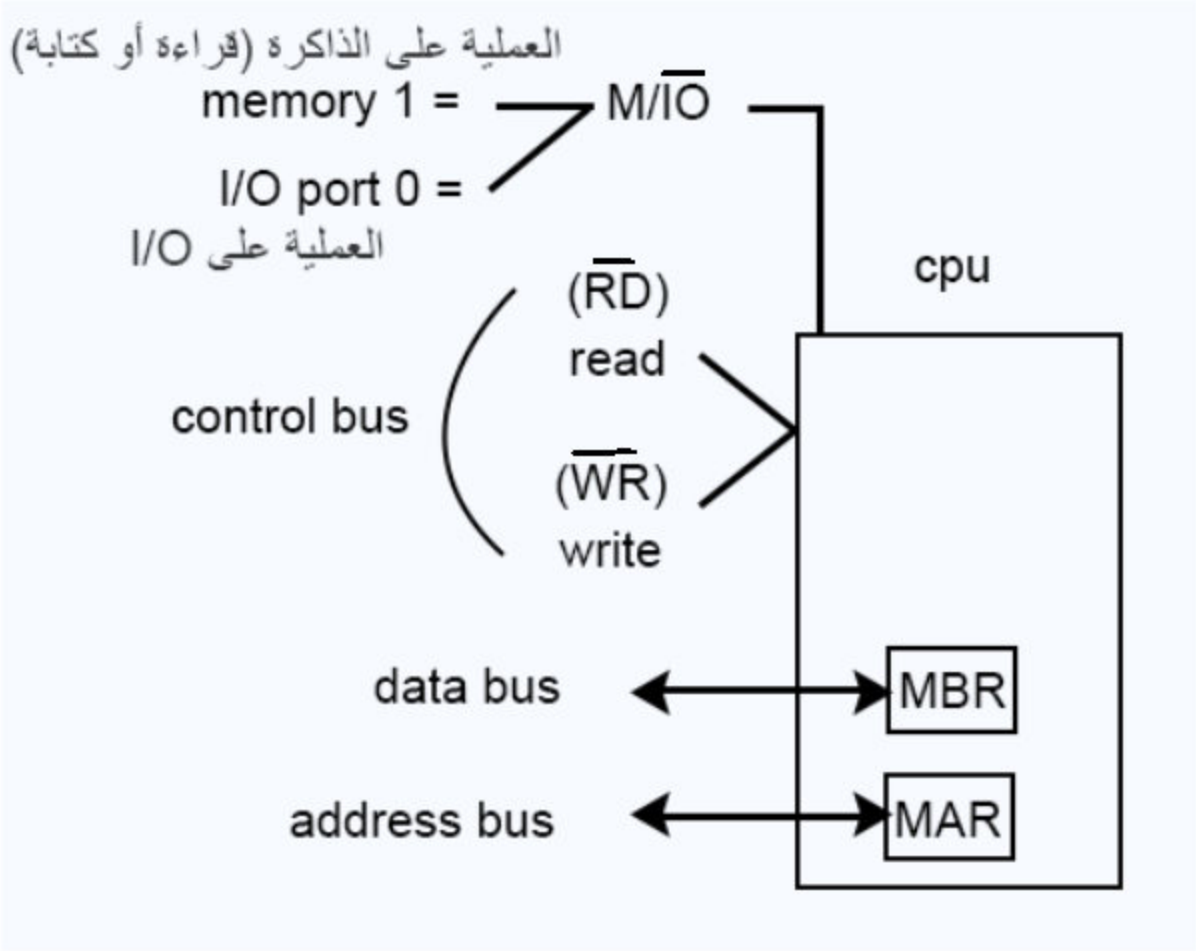
CS	WE	OE	
1	x	x	الذاكرة في حالة الممانعة العالية (Hiz) الذاكرة تحافظ على محتواها
0	0	1	Read
0	1	0	Write
0	0	0	x

المحاضرة الثانية

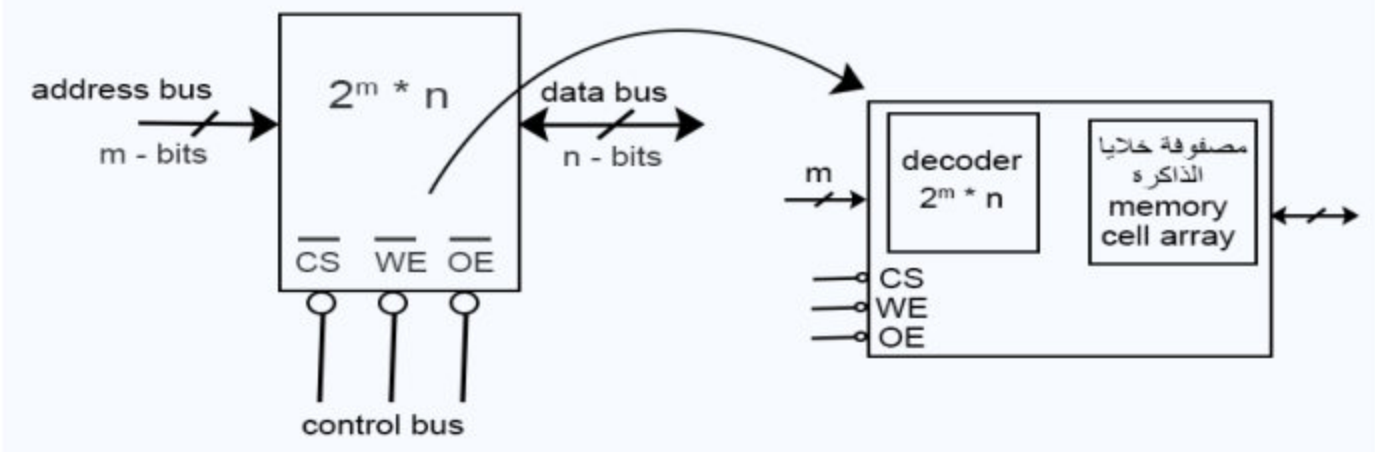
تتمة في البنية الداخلية للذواكر

SRAM ليست ك cache

التوصيل بين SRAM و CPU

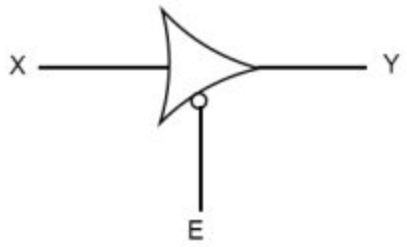


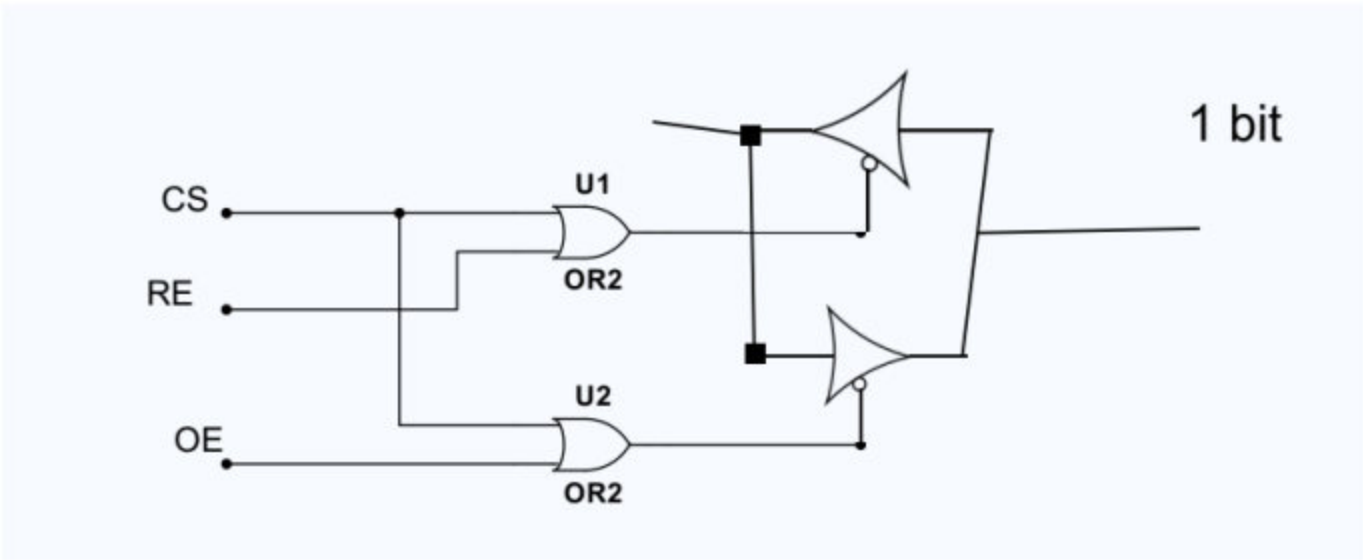
الذاكرة SRAM تحوي decoder مبين بالشكل



رسم يوضح مبدأ عمل الذاكرة

E	Y
0	X
1	Hi-z (z0 = ∞)

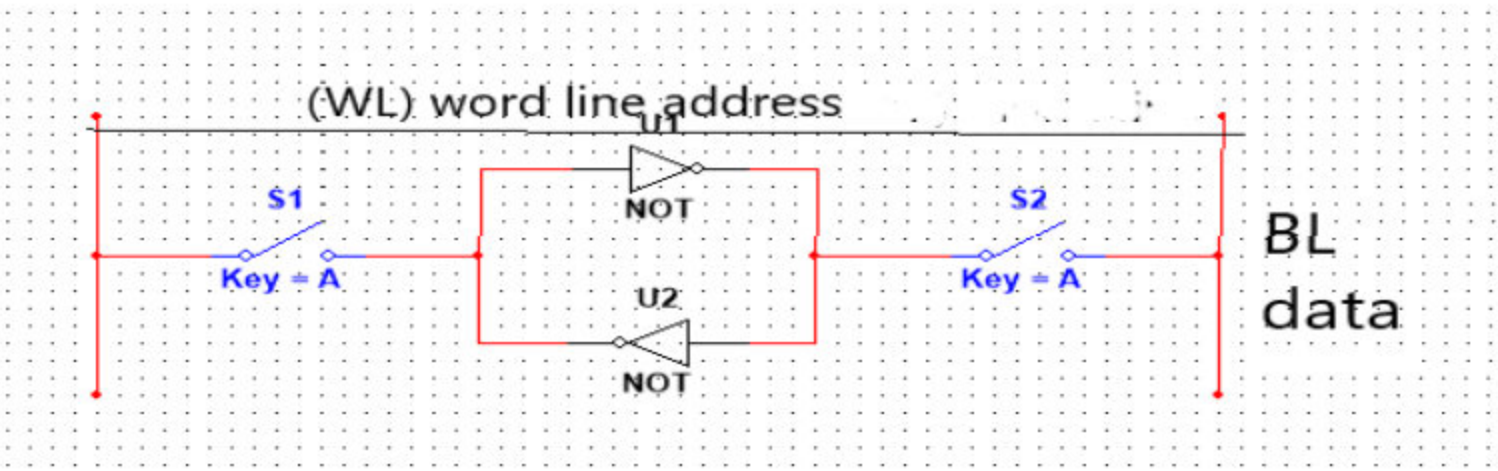




شكل خلية واحدة من خلايا المصفوفة ضمن الذاكرة SRAM

الخلية الواحدة تحتاج من 4 ترانزستورات إلى 6

الدارة التالية تشير إلى خلية واحدة تخزن 1 بت وتتألف من 6 ترانزستورات

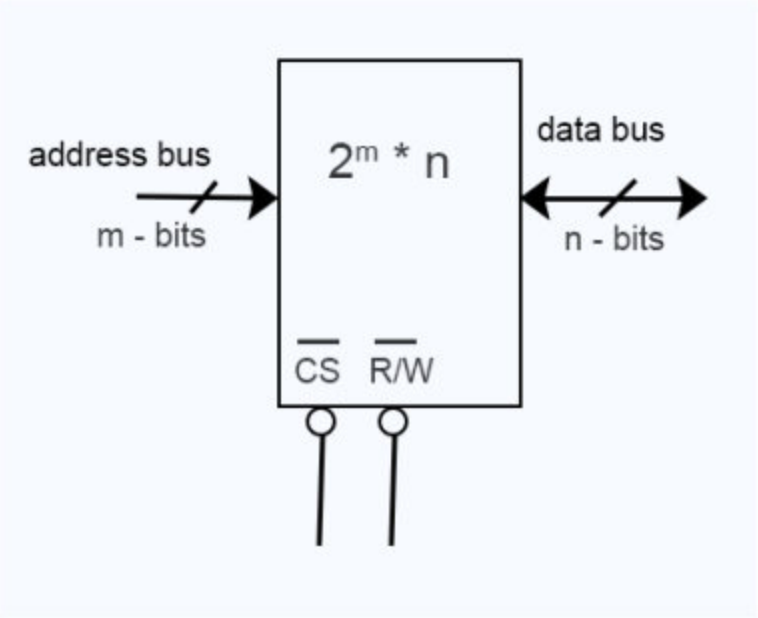


BL : bit line كل عاكس هو ترانزستور

ملاحظة :

إذا كانت الكلمة مؤلفة من 8 بتات فمصفوفة الخلايا تكون مؤلفة من 8 من الدارة السابقة وهذا

شكل ثان لذاكرة SRAM : تم دمج خط RE و خط WE في خط واحد هو : R/W



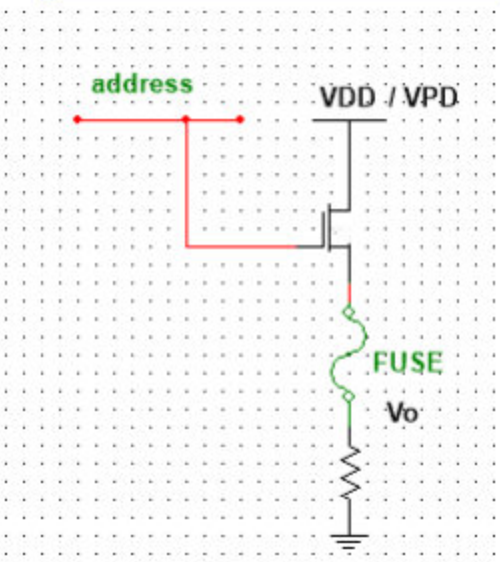
عندما تكون

R/W = 1 تكون إشارة قراءة read

R/W = 0 تكون إشارة كتابة write

السؤال يأتي : ارسم البنية الداخلية لذاكرة SRAM أو ارسم خلية واحدة من مصفوفة الخلايا في الذاكرة SRAM

ذاكرة ال ROM



شكل ال cell فيها :

شكل الخلية / cell في كل من PROM و MROM :

VDD : جهد العمل الطبيعي مثلا 5v

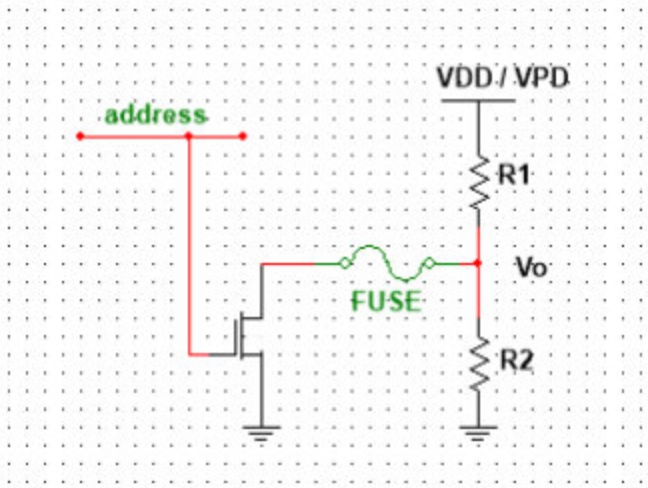
VPD : جهد البرمجة 10 - 20v

من المعمل :

تخزن ال cell "1"

بعد البرمجة : لا يتحمل ال fuse الجهد العالي فتفصل الدارة

لذلك تخزن ال cell "0"



شكل ثان للخلية : تغير مكان ال fuse

Vo = VDD (R2 / R1+R2)

غالبا تكون R2 >> R1

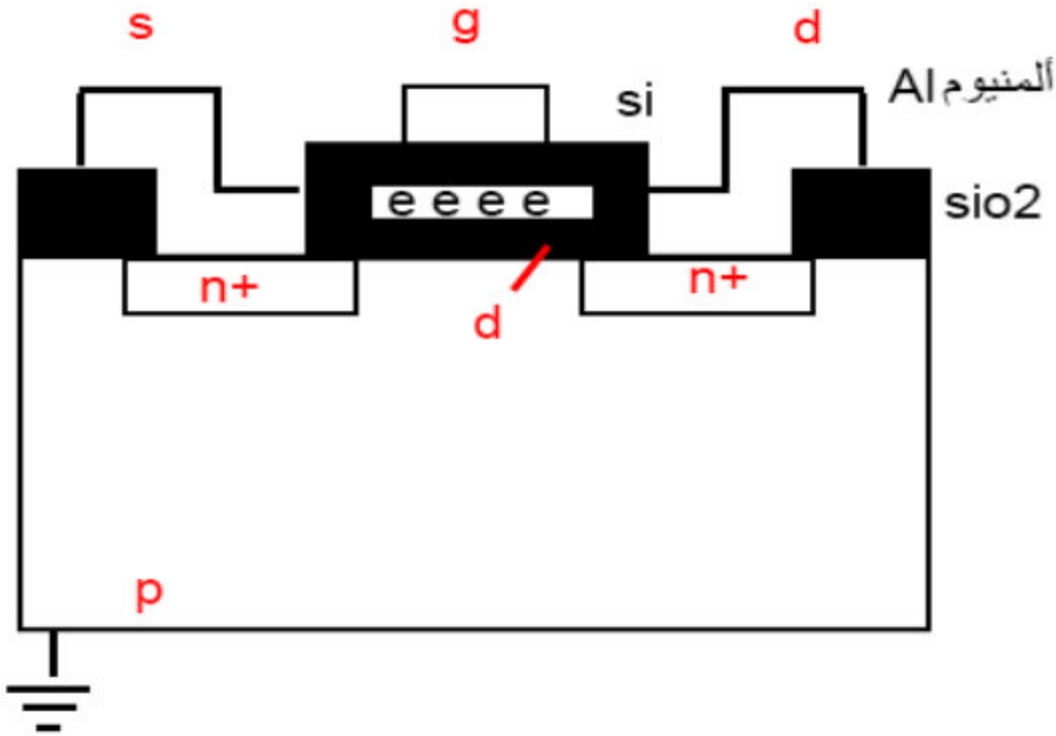
Vo = VDD

من المعمل : يتم تخزين "0"

بعد البرمجة : يتم تخزين "1" حيث عند تطبيق الجهد العالي يسلك الطريق الأسهل وهو الفيوز وينزل كامل الجهد على R2

شكل خلية كل من EPROM و EEPROM و Flash

floating gate

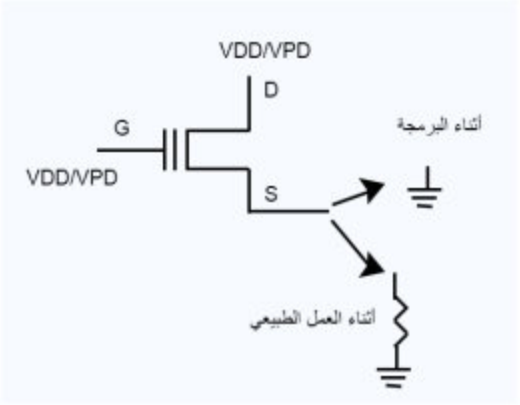


من المعمل وعند تطبيق جهد العمل الطبيعي VDD ==> Tn(on)

Vo = "1"

بعد البرمجة : وعند تطبيق جهد 10 فولت مثلا يكون Tr(off)

Vo = "0"



المحي : هو إعطاء طاقة لإرجاع الالكترونات في floating gate

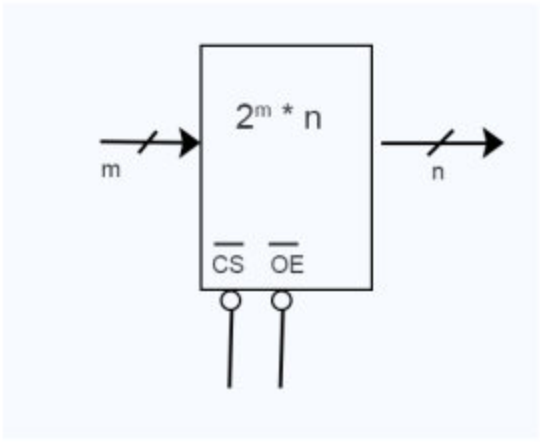
حيث تكون الطاقة في : 1) EEPROM جهد (عن طريق الكهرباء حيث يطبق جهد سالب عالي على G

- EPROM ضوء
- flash عن طريق الكهرباء وتقليل سماكة طبقة الأوكسيد

في Flash تكون ال d صغيرة جدا ==> المحي والكتابة سريعة جدا

Flash	EEPROM
كتابة سريعة	كتابة بطيئة
المحي كلمة كلمة أو كتلة	المحي يمكن كلمة كلمة
تصنع من Nand أو Nor	
تتحمل 10k عملية كتابة	تتحمل 100k عملية كتابة

شكل ROM داخليا عدا كل من (EEPROM و Flash)



ذاكرة Flash

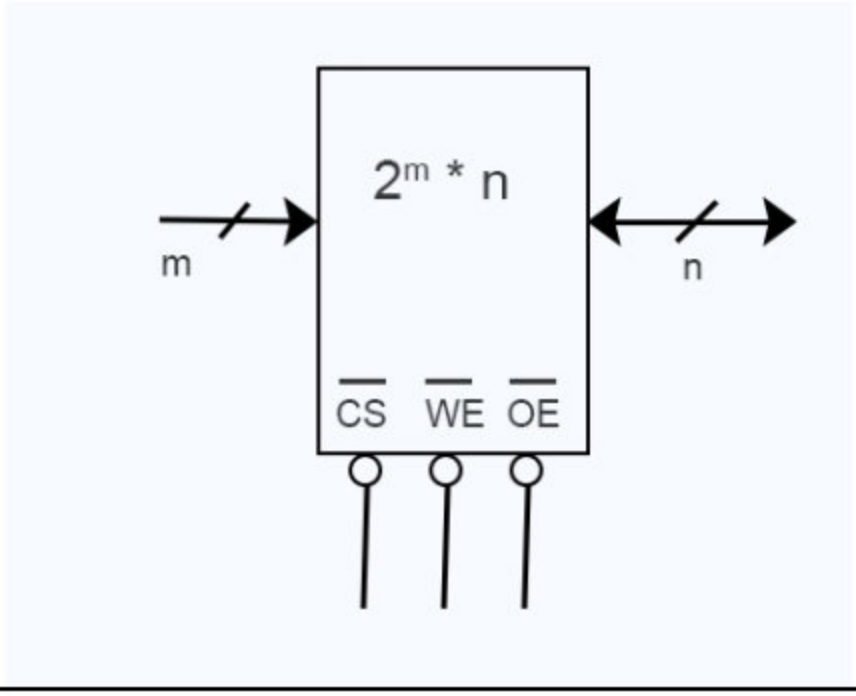
Nand

المحي كتلة

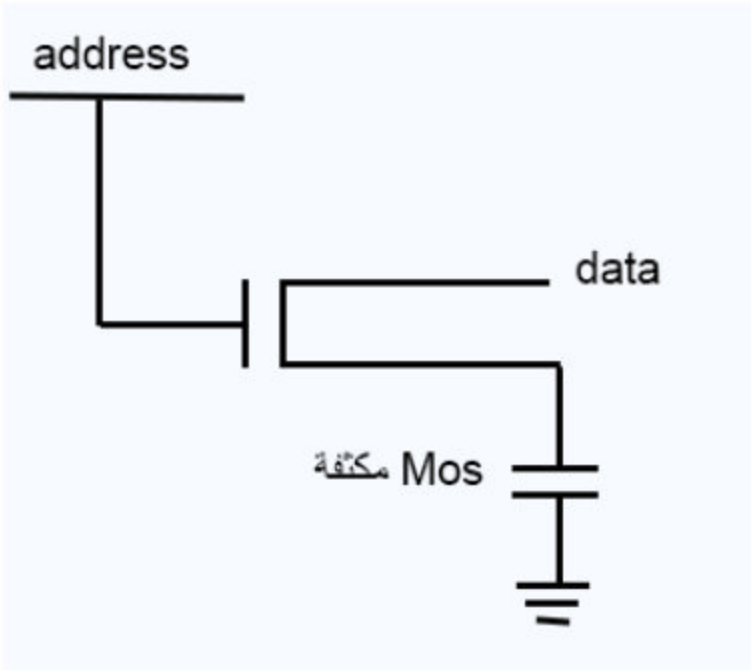
Nor

المحي كلمة كلمة

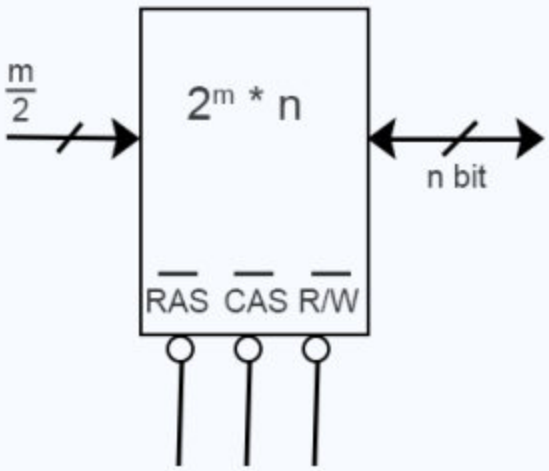
شكل ال Flash



DRAM Cell



شكل الذاكرة DRAM :



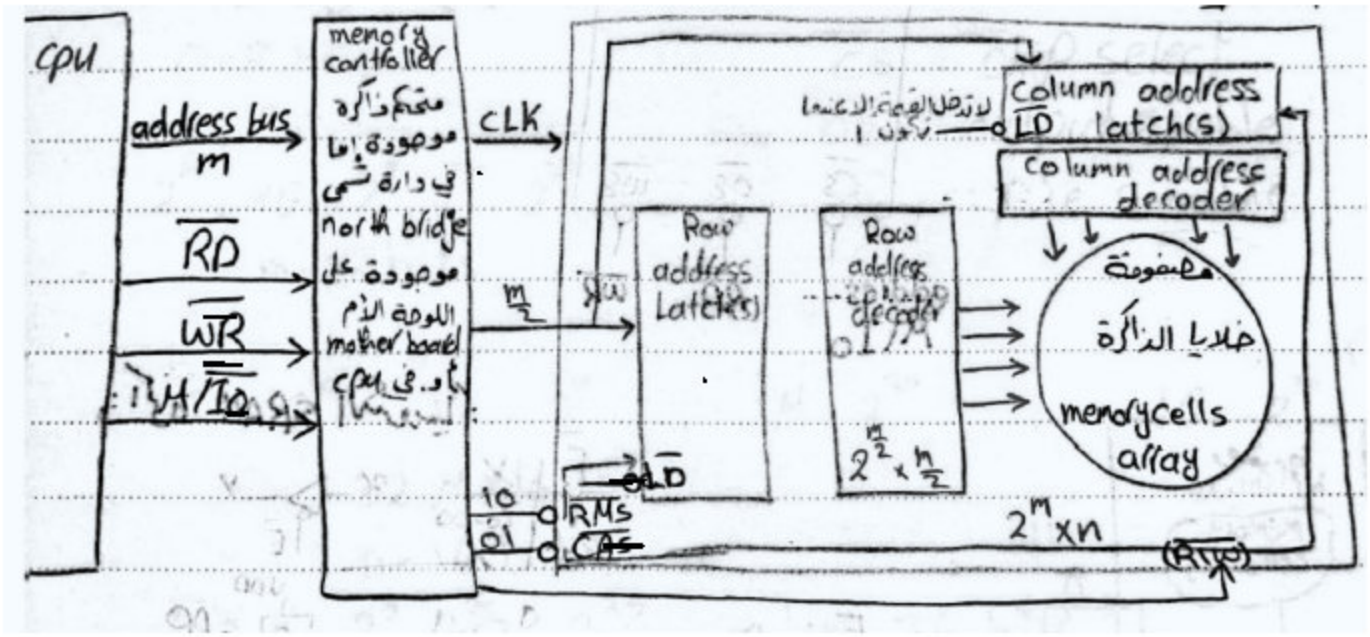
RAS : row address strobe نبضة مسك عنوان السطر

CAS : column address strobe نبضة مسك عنوان العمود

m كبيرة جدا بالنسبة لذواكر DRAM

لذلك يقسم العنوان إلى قسمين لتخفيف ء

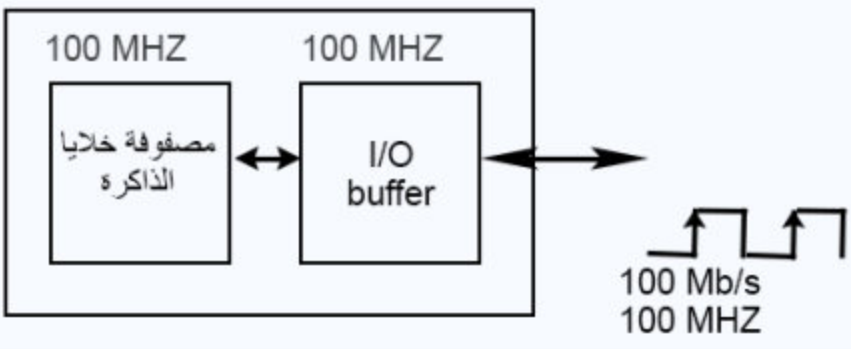
البنية الداخلية للذاكرة :



بعض أنواع DRAM

- SDRAM (synchrous DRAM) : يعمل عند الحافة الصاعدة أو الهابطة لنبضة الساعة تتصف بالبطأ

السرعة : 100 Mb/s

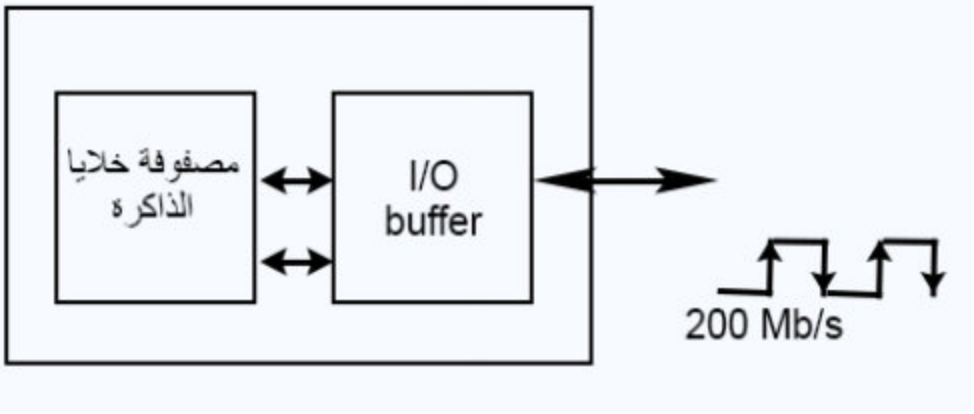


- DDR (double data rate)

2 * clock rate

(2n - bit) 2-bit prefetch buffer

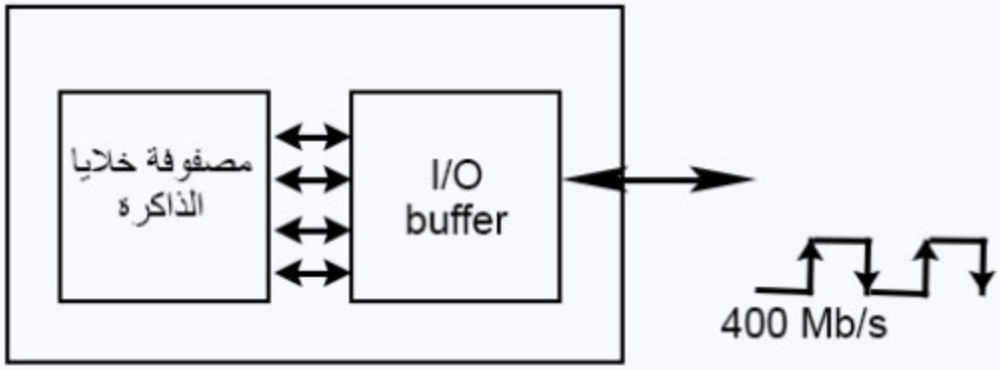
يقرأ كلمتين كل
منهما n بت
جهد التغذية : 2.5v



- DDR2

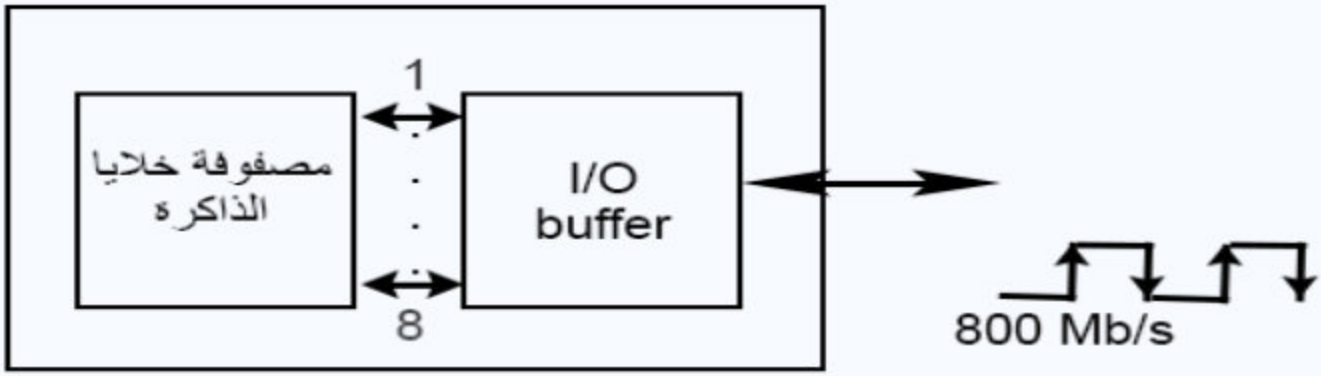
(4n-bit) 4-bit prefech buffer

جهد التغذية : 1.8v



- DDR3

8-bit prefech buffer جهد التغذية : 1.5v


مصفوفة خلايا
الذاكرة
1
8
I/O
buffer
800 Mb/s

المحاضرة الثالثة

مواصفة ذاكرة DDRAM

معدل نقل المعطيات بين ال cpu والذاكرة الرئيسية MM

- مقياس أول : DDR3_1250 : سرعة الساعة الفعلية effective clock rate
  1250 Mb/s أو 1250 MHZ أو 1250 MT/s
- مقياس ثان : pc3_1000

عرض المجال النظري theoretical band width

Data rate = 2 clock rate

سرعة نقل البيانات بين ال cpu و شريحة الذاكرة board

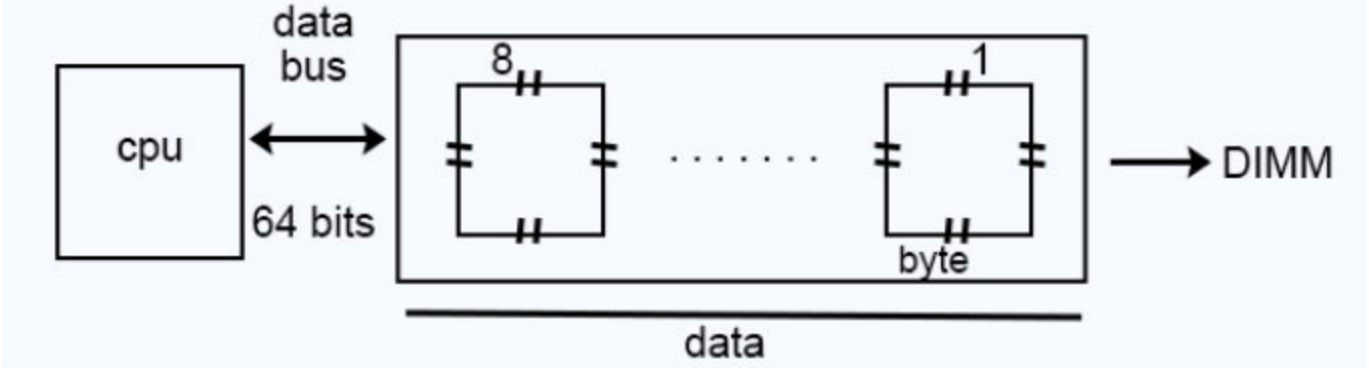
$$1250 * 8\ byte = 10000\ Mb/s$$

السبب أن أغلب ال cpu لديها data bus (64 bits)

عادة يتم وضع دارات ذواكر (DDR) IC على شرائح أشهرها شريحة

DIMM (dual in line memory module)

عرض الكلمة لكل دارة (IC) هو 8bits أو byte



عادة يتم وضع 8 ذواكر (IC) 8 * 8 = 64 bits أو 8 byte

64 bits لأن أغلب المعالجات لديها data bus 64 bits

يوجد شرائح أخرى مثل : SIMM (الذواكر على وجه واحد وال ping نفسها على الوجهين)

بينما DIMM (ذواكرة على وجهين أو وجه واحد وال ping مختلفة على الوجهين)

كما ويوجد نوع RIMM (Rambus) وهو الأغلى والأسرع

duol channel

يتم توصيل 2 DIMM بنفس الوقت مع المعالج ويتم نقل 128 bits من الذاكرة إلى المعالج عن طريق متحكم الذاكرة

إذا كان لدينا 32 address bus يمكننا عنونة : $2^{32} = 4\ GB$

وبالتالي عند وضعها بال DIMM يكون لكل وحدة 0.5 GB

عادة تقرأ المعالجات كلمات بأطوال مختلفة من الذاكرة bits(8,16,32,64,...)

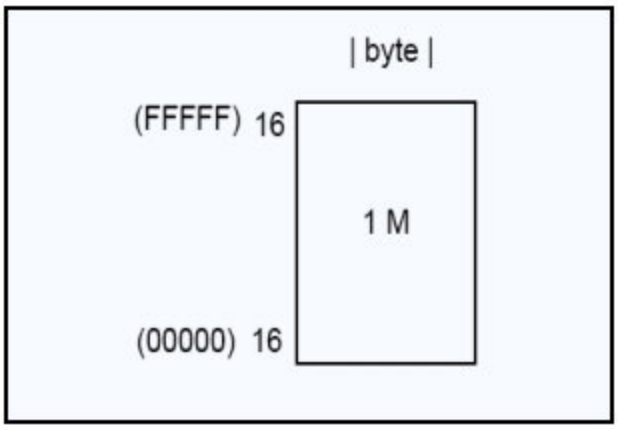
بعض القوانين التي يجب حفظها لتطبيقها في الأسئلة :

- ذاكرة النظام (system memory - memory system) : $\mathbf{2^{address\ bus}}$ للمعالج
- عدد ال الذواكر أو ال ICS : $\frac{data\ bus}{8}$
- حجم الذاكرة : $\frac{2^{address\ bus}}{\frac{data\ bus}{8}}$

أي ناتج قسمة 1 على 2

مثال 1 :

Address bus	Data bus	نوع المعالج
20 bits	8 bits	8080
ذاكرة النظام : $2^{20} = 1\ MB$ عدد ICS : $\frac{8}{8} = 1\ Ic$ حجم الذاكرة : $\frac{1MB}{1} = 1MB$		



شكل ال ICS :

المطلوب معرفة أول عنوان وآخر عنوان

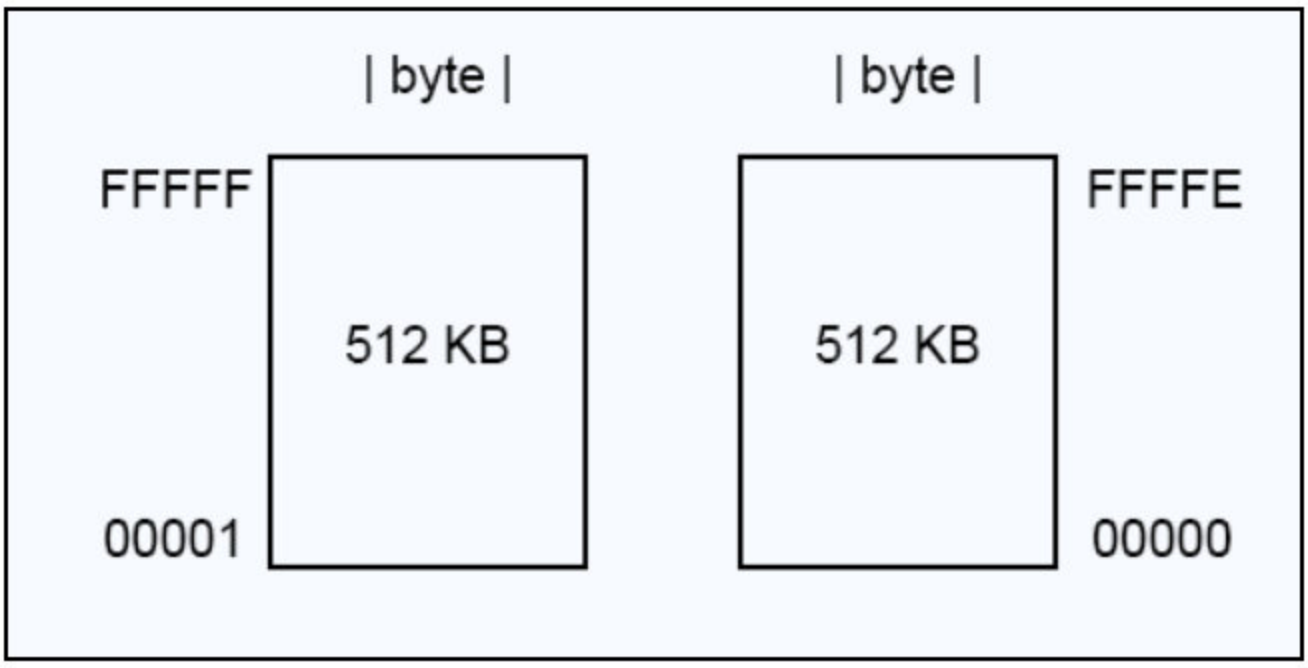
ويكون العنوان بالنظام السداسي عشر (HEX)

لدينا 20bits للعنوان ==> 5 خانات للعنوان بالنظام hex

مثال 2 :

نوع المعالج	Data bus	Address bus
8086	16 bits	20 bits

- ذاكرة النظام : $2^{20} = 1\ MB$
- عدد Ics : $\frac{16}{8} = 2\ Ics$
- حجم الذاكرة : $\frac{1MB}{2} = 512\ KB = 0.5\ MB$



8086 يقرأ أو يكتب 8 bit أو 16 bit مع الذاكرة

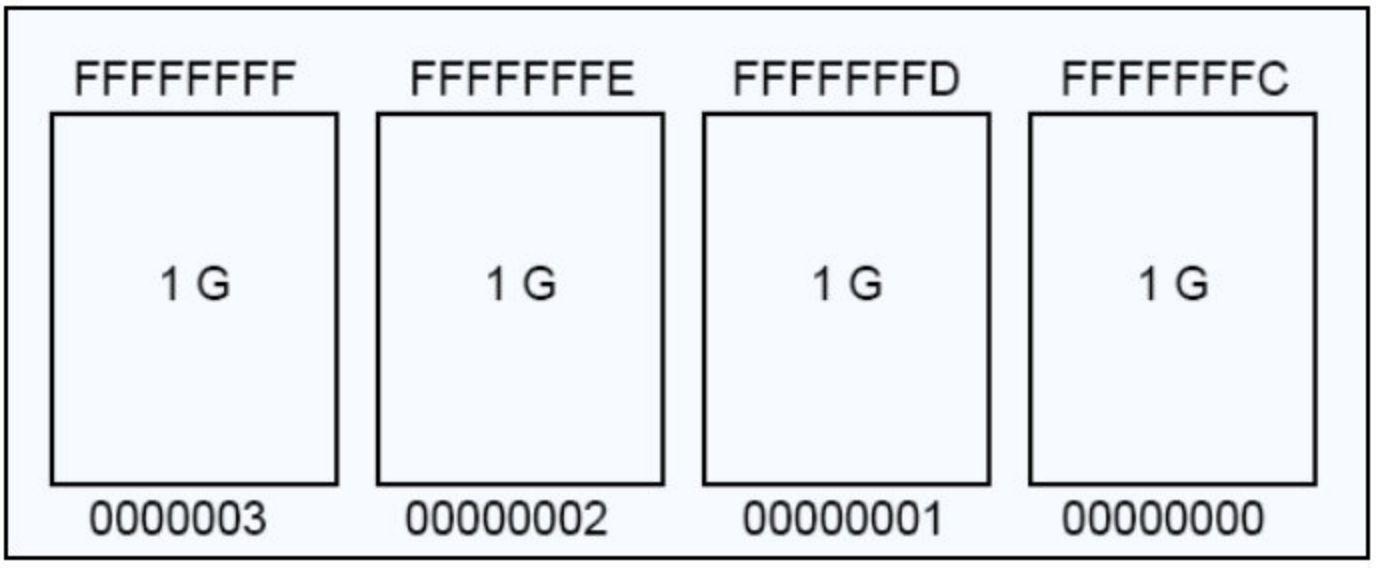
20 binary == > 5 Hex (÷4)

عنوان الكلمة هو عنوان الأدنى فيها

مثال 3 :

نوع المعالج	Data bus	Address bus
80386	32 Bits	32 Bits

- ذاكرة النظام : $2^{32} = 4\ GB$
- عدد Ics : $\frac{32}{8} = 4\ Ic$
- حجم الذاكرة : $\frac{4GB}{4} = 1\ GB$



يقرأ أو يكتب 8 أو 16 أو 32 bit

32 binary == > 8 Hex

لدينا 32 bit address bus أي لدينا 8 خانات بالسداسي عشر لذلك بدأ أول عنوان 00000000

0000 0000 0000 0000 0000 0000 0000 0000

0 0 0 0 0 0 0 0

ملاحظة هامة : ليس دائما آخر خانة بالعنوان الأخير F بالضرورة
فإذا كان لدينا 18bit address bus مثلا
يكون لدينا العنوان الأخير

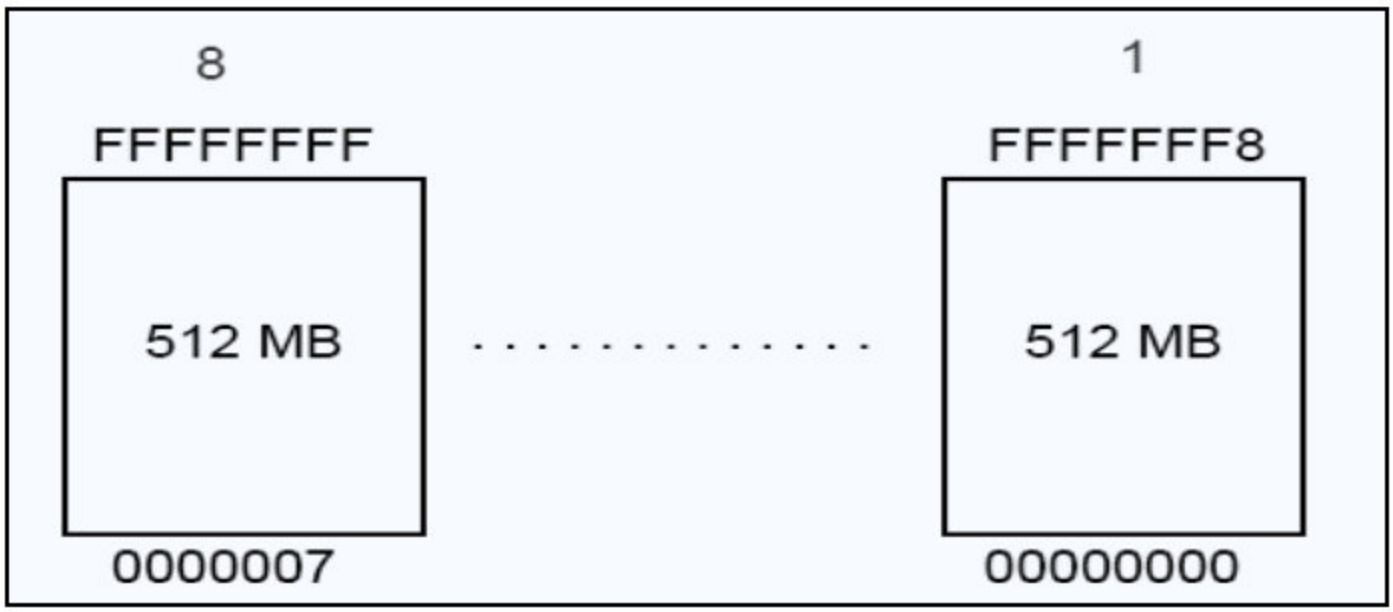
0011 1111 1111 1111 1111

3 F F F F

مثال 4 : معالج Pentium

نوع المعالج	Data bus	Address bus
Pentium	64 bits	32 bits

- ذاكرة النظام : $2^{32} = 4\ GB$
- عدد ال Ics : $\frac{64}{8} = 8\ Ic$
- حجم الذاكرة : $\frac{4GB}{8} = 0.5\ GB = 512\ MB$



يقرأ أو يكتب 8 أو 16 أو 32 أو 64 bit

- عملية القراءة : 1) نضع address

2) يعطي أمر Read(RD)

3) Access Time (for read) الزمن الأعظمي الفاصل بين وضع العنوان وظهور المعطيات على خطوط معطيات الخرج

- عملية الكتابة : 1) نضع address

2) ثم data

3) ثم يعطي أمر write (WR)

4) يظهر لدينا Access Time (for write) الزمن الفاصل بين وضع العنوان وتخزين المعطيات في الذاكرة

- Cycle time

الزمن الفاصل بين لحظة الوصول الأولى للذاكرة (قراءة أو كتابة) ولحظة الوصول الثاني (قراءة أو كتابة)

المشاكل :

1) cycle time = Access time + زمن انتهاء الحالات العابرة

القراءة لذاكرة ال DRAM هي قراءة مدمرة (يلزم عند القراءة إعادة كتابة الموقع نفسه الذي تم قراءته)

2) cycle time > access time

عند الوصول لموقع واحد في الذاكرة بعنوان واحد

3) cycle time < access time

عندما تعمل الذاكرة بنظام الرشقة burst mode الوصول لأكثر من موقع بعملية واحدة بعنوان واحد ثم تتم زيادته بشكل آلي

1) data rate / band width / throughput

معدل نقل المعطيات من وإلى الذاكرة

$$data\ rate = \frac{1}{cycle\ time}$$

مسألة دورة :

صمم نظام ذاكرة يتشكل من

ذاكرة EPROM 8M X 16 مؤلفة من قطع 256K X 2

وذاكرة SRAM 1024K X 16 مؤلفة من قطع 32KB

بحيث تبدأ ذاكرة EPROM من العنوان 800000H

وذاكرة SRAM من العنوان 7000000H

ضع جدول لتوزع العنونة (خريطة الذاكرة) باستخدام طريقة العنونة الكاملة وبفرض أن المعالج الذي سيربط معه نظام الذاكرة يملك 24 خط عنونة و 16 خط معطيات

الحل :

تحليل السؤال :

ذاكرة EPROM 8M X 16 (تعني data bus)

تبدأ من العنوان 800000H (تعني address bus 24bit)

حرف ال H يعني أن نظام hex سداسي عشر

وكذلك بالنسبة إلى SRAM

العنونة الكاملة : تعني استخدام كل خطوط العنونة الخارجة من المعالج

ضع جدول لتوزع العنونة : يعني أول عنوان وآخر عنوان بكل ذاكرة

بالامتحان :

- يتم تغير address bus & data bus

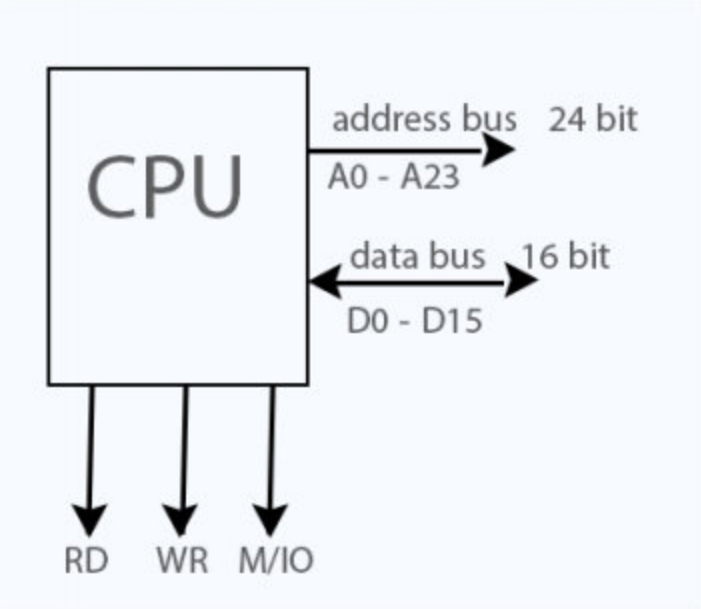
- وقد لا يذكر عدد عدد خطوط العنونة والمعطيات فنستنتجهم من نص السؤال
  800000 العنوان مكون من 6 خانات بالنظام السداسي عشر:

  8 0 0 0 0 0

  1000 0000 0000 0000 0000 0000

  أي لدينا address bus 24bit

- التصميم يأتي حصرا لذواكر EPROM و SRAM



نبدأ بالذاكرة EPROM :

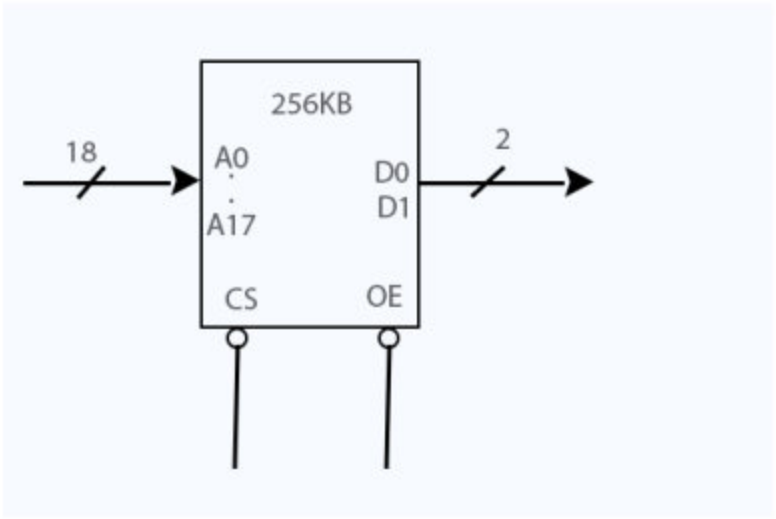
$$\frac{8M}{256k} * \frac{16}{2} = \frac{2^{13}}{2^{8}} * 8 = 2^{5} * 8 = 32 * 8$$

8 : عدد القطع بالعرض (تكبير حجم الكلمة)

32 : عدد القطع بالطول (تكبير السعة)

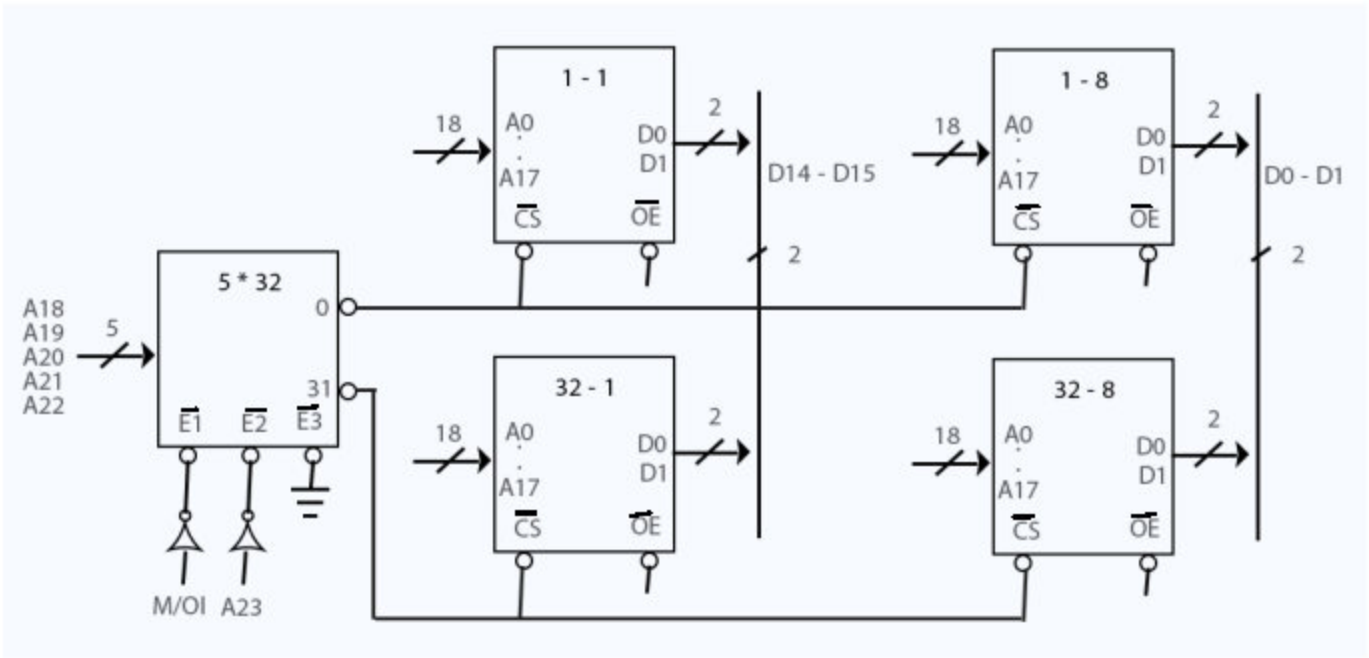
للتوضيح :

$$8M = 2^{3} * 2^{20} = 2^{23}$$
$$256k = 2^{8} * 2^{10} = 2^{18}$$
$$\frac{2^{23}}{2^{18}} = 2^{5} = 32$$



ملاحظات : لا يوجد بذاكرة EPROM خط WE

مطلوب منا رسم أول خط وآخر خط فقط

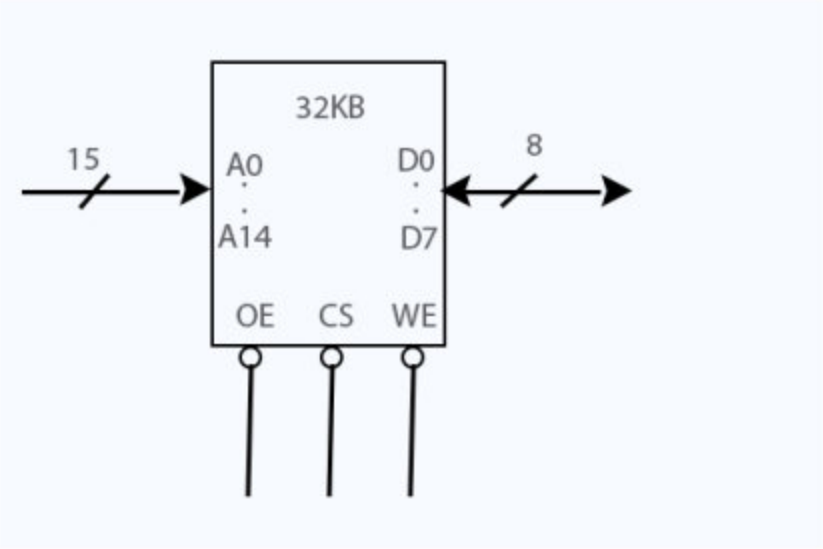


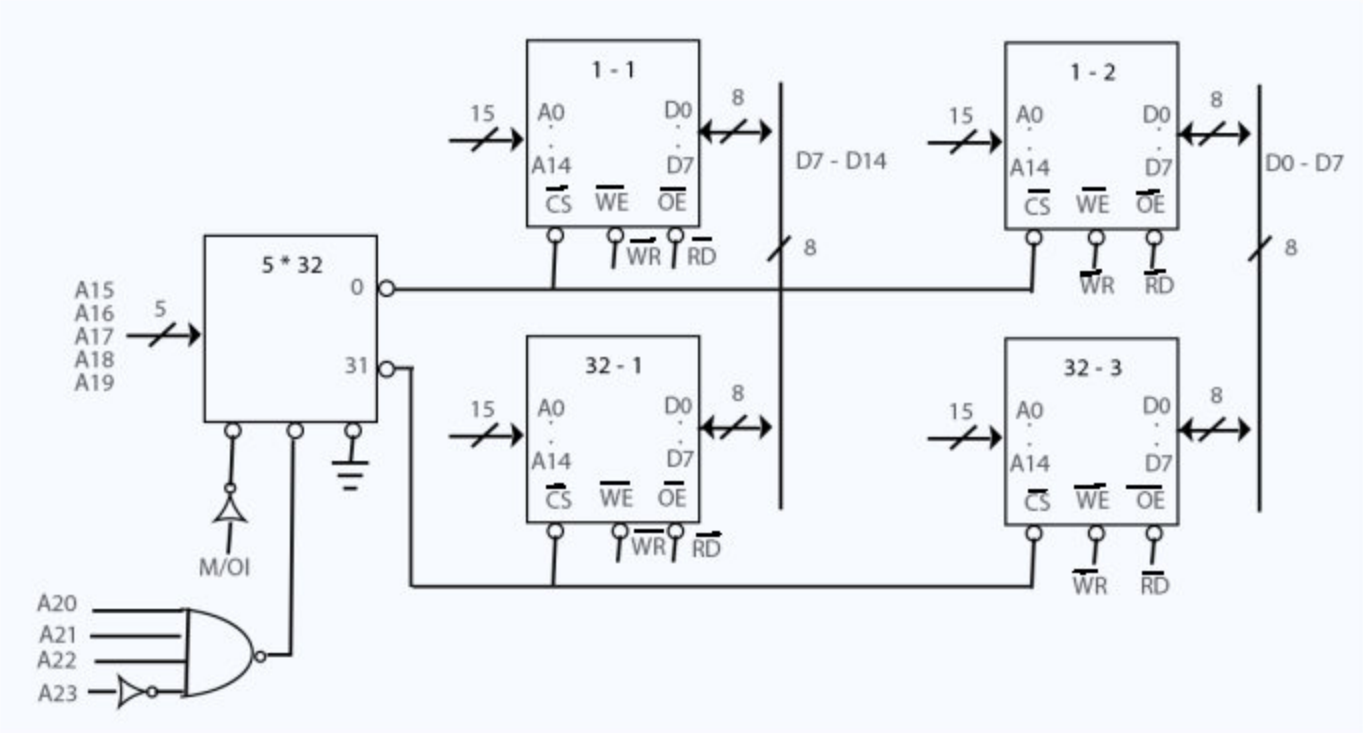
خريطة الذاكرة :

A23	A22 A18	A17 ..... A0	
1	000 00	00 0000 0000 0000 0000	800000
1	111 11	11 1111 1111 1111 1111	FFFFFF

ثانيا : ذاكرة SRAM

$$\frac{1024k}{32k} * \frac{16}{8} = 32 * 2$$
$$32k = 2^5 * 2^{10} = 2^{15}$$





خريطة الذاكرة :

A23 A20	A19 A15	A14 ..... A0	
0111	0000 0	000 0000 0000 0000	700000
0111	1111 1	111 1111 1111 1111	FFFFFF

سؤال إضافي : هل يمكن زيادة سرعة الذاكرة ؟

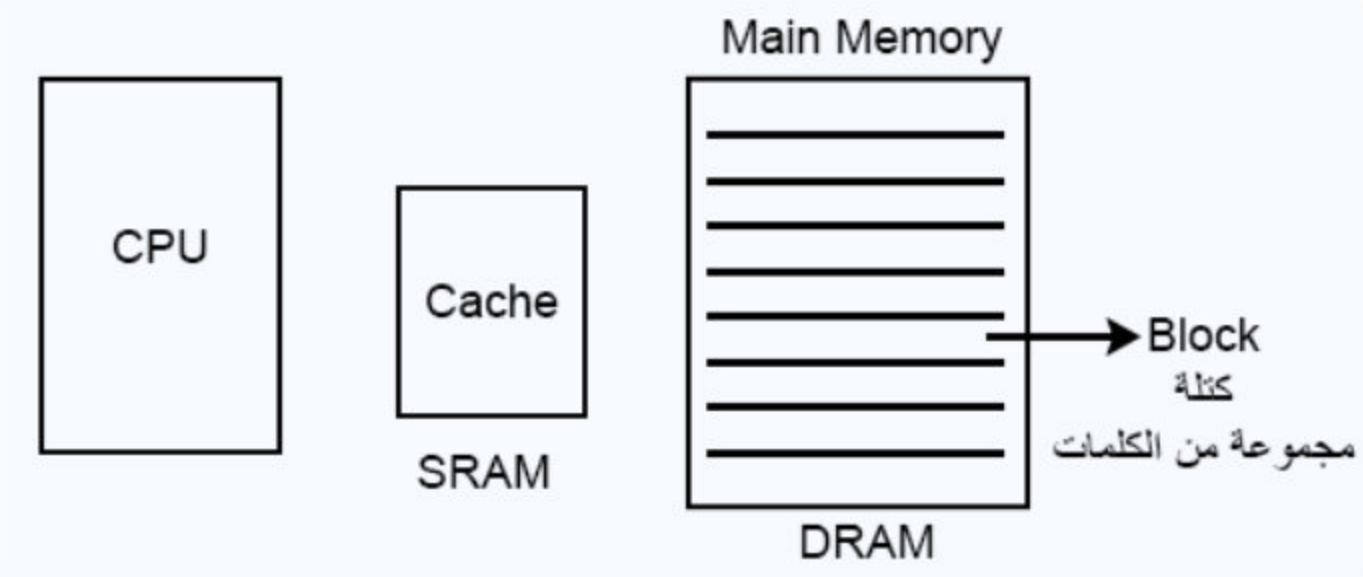
$$\frac{1024k}{32k} * \frac{16}{8} = 32 * 2$$

$$32KB <==> 32k * 8$$

$$2^5\ 2^{10} = 32k = 2^{15}$$

المحاضرة الرابعة

ذاكرة الكاش cache



ملاحظات عن الرسم: سرعة ال DRAM : 100ns وسرعة ال SRAM : ns(1-10)

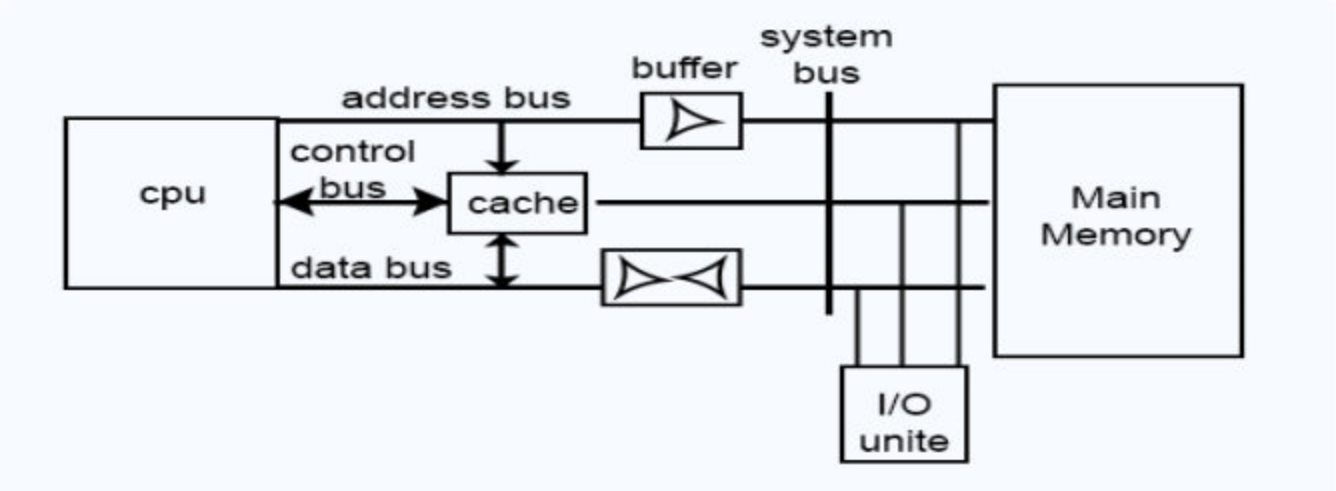
مقدمة : يتم نقل كامل الكتلة التي يريدها ال cpu إلى الكاش دون تجزئة

عند تنفيذ تعليمات برنامج ما

غالبا تكون التعليمات في مواقع تحت بعضها إلا إذا وجد تعليمة go to

في البداية تكون الكاش فارغة فعند البحث عن كلمة يتم البحث عليها في الذاكرة الرئيسية وعندنا يجدها ينقل الكتلة كاملة من الذاكرة الرئيسية ويضعها في الكاش

بعد ذلك يذهب إلى الكاش فإن وجد الكلمة المطلوبة جلبها من الكاش وإلا عاد إلى الذاكرة الرئيسية



ضمن ال cache يوجد متحكم controller - التحكم بالكاش هو HW (المبرمج غير قادر على التحكم فيه)

آلية العمل : يبحث المعالج cpu عن الكلمة word التي يحتاجها في cache فإن وجدها كانت العملية سريعة وإذا لم يجدها يتجه للذاكرة الرئيسية MM ويحضر كامل الكتلة block التي تحتوي الكلمة وتخزنها في cache وبنفس الوقت على الوقت على التوازي تحضر الموقع الذي تبحث عنه وتضعه في cpu

الهدف من ذاكرة cache : الحصول على سرعة عالية للذاكرة، سعة كبيرة، تكلفة أقل، تعتبر ذاكرة cache وسيطا بين ال cpu و MM

مثلا : core i7 64 bytes يوجد 3 كاش I1,I2,I3

D-cach	Data	32 KB * 2	L1	معطيات
I-cach	Instraction			تعليمات
		256 KB	L2	
		8MB	L3	

ملاحظة : إذا زاد عدد الكاش عن زادة الفعالية فإذا وصل العدد إلى 4 كاش انخفضت الفعالية حسب خوارزميات (إذا لم يجد الكملة التي يبحث عنها في الكاش الأولى ذهب إلى الثانية ثم الثالثة وهذا يستغرق وقت أطول)

عناصر تصميم الكاش :

- تابع التقابل : mapping function (طرق تنظيم ال cache)

بفرض n عدد خطوط عنونة المعالج address bus == > عدد الكلمات التي يستطيع المعالج عنونتها في الذاكرة الرئيسية 2^n كلمة

تقسم الذاكرة الرئيسية إلى كتل (M كتلة) وعددها يساوي $M = 2^s$ كتلة حيث أن S عدد البتات التي تدل على رقم الكتلة

كل سطر في الكاش يتسع لكتلة واحدة

كل كتلة تتألف من 2^w كلمة word حيث w عدد البتات اللازمة لعنونة الكلمات ضمن الكتلة الواحدة (عدد البتات التي تدل على رقم الكلمة في الكتلة)

$$2^s * 2^w = 2^n <==> s + w = n$$

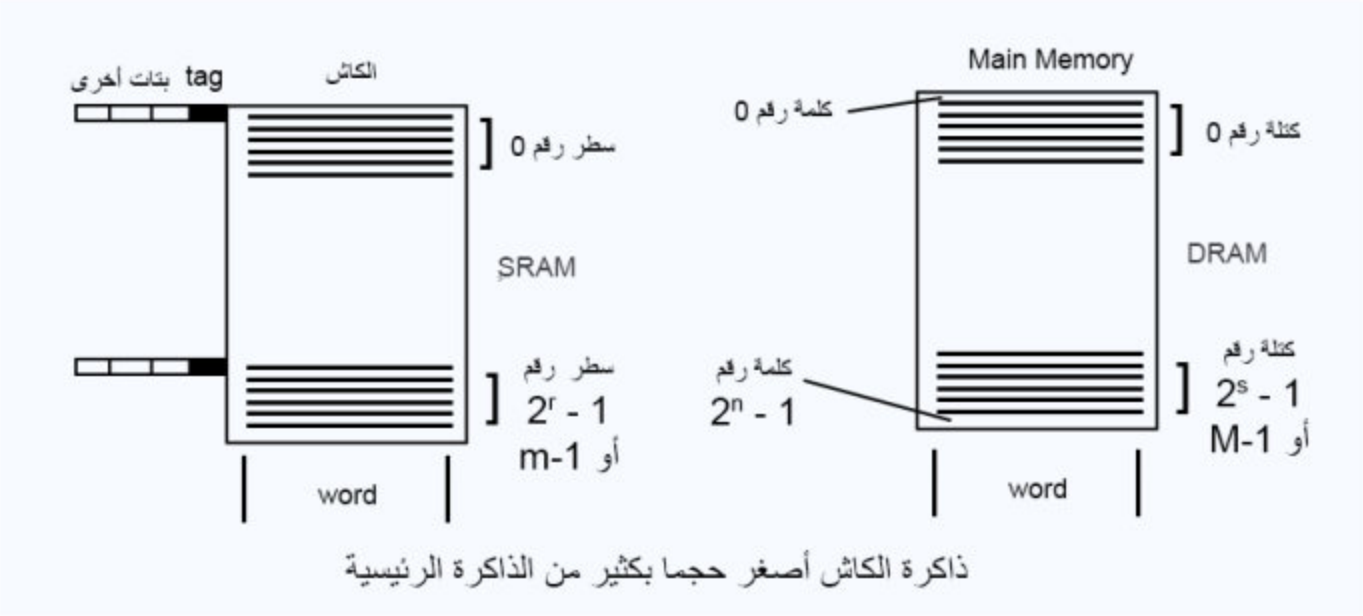
شكل عنوان الذاكرة الرئيسية الذي يصدره المعالج:

n

s	w

تقسم الكاش إلى أسطر line عددها $m = 2^r$ حيث r هو عدد البتات التي تدل على رقم السطر

حجم الكتلة = حجم السطر كمعطيات



ذاكرة الكاش أصغر حجما بكثير من الذاكرة الرئيسية

شكل الكاش عبارة عن أسطر بالشكل :

كتلة المعطيات + tag + بتات أخرى
.
.
.
.
كتلة المعطيات + tag + بتات أخرى

السطر الواحد :



Tag : هو جزء من عنوان الذي يرسله المعالج لعنونة الذاكرة الرئيسية ويستخدم للدلالة على الكتلة الموجودة حاليا في السطر حيث أن m << M (M عدد الكتل و m عدد الأسطر) وبالتالي فإن كل سطر هو مكان محتمل لأكثر من كتلة في لحظة معينة يوجد في هذا السطر كتلة واحدة

من البتات الأضافية : use,update,valid

مثال : الذاكرة الرئيسية فيها 16 كتلة والكاش فيها 4 أسطر وبالتالي كل سطر هو مكان محتمل ل 4 كتل

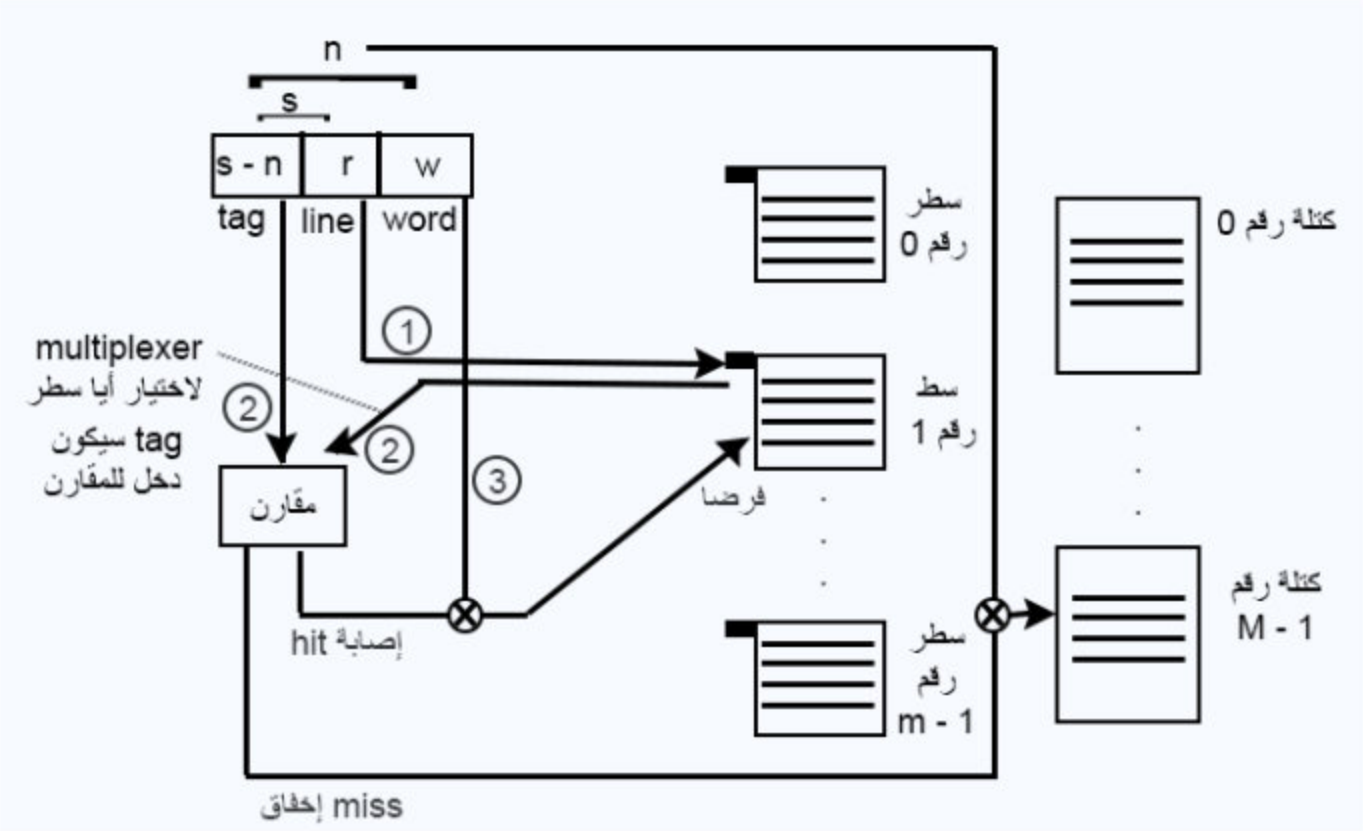
ال tag يستخدم لمعرفة أي كتلة من هذه الكتل موجودة في السطر حيث كل كتلة لها tag مختلف للتمييز بين الكتل.

في أي سطر ستوضع الكتلة ؟ الجواب 3 طرق (تابع التقابل):

- Direct mapping التقابل المباشر: لكل كتلة سطر محدد توضع فيه حسب العلاقة رقم السطر بالكاش $i = j \bmod m$

حيث j رقم الكتلة بالذاكرة الرئيسية و m عدد الأسطر بالكاش

Mod : أي باقي القسمة



أولا : نبحث في السطر، ثانيا : نستخدم دارة مقارن ()، ثالثا : إذا كانت إصابة (خرج المقارن 1) نعود إلى نفس السطر ونأخذ منه كلمة ونضعها في المكان المناسب، رابعا : إذا كانت إخفاق (خرج المقارن 0) يذهب إلى الذاكرة الرئيسية ونأخذ s + w ونضعها في المكان المناسب.

المزايا : نحتاج لمقارن واحد لكل كاش

المساوئ : كل سطر مكان محتمل لأكثر من كتلة وبفرض احتجنا لكتلتين من هذه الكتل (التي لها نفس السطر) سنجبر على استبدال الكتلة السابقة بالكتلة الحالية مع أن بقية الكاش ممكن أن تكون فارغة

ملاحظات عن الرسم :

- $n = s + w$
- الإصابة في الكاش : تعني أن الكتلة التي نبحث عنها موجودة في الكاش

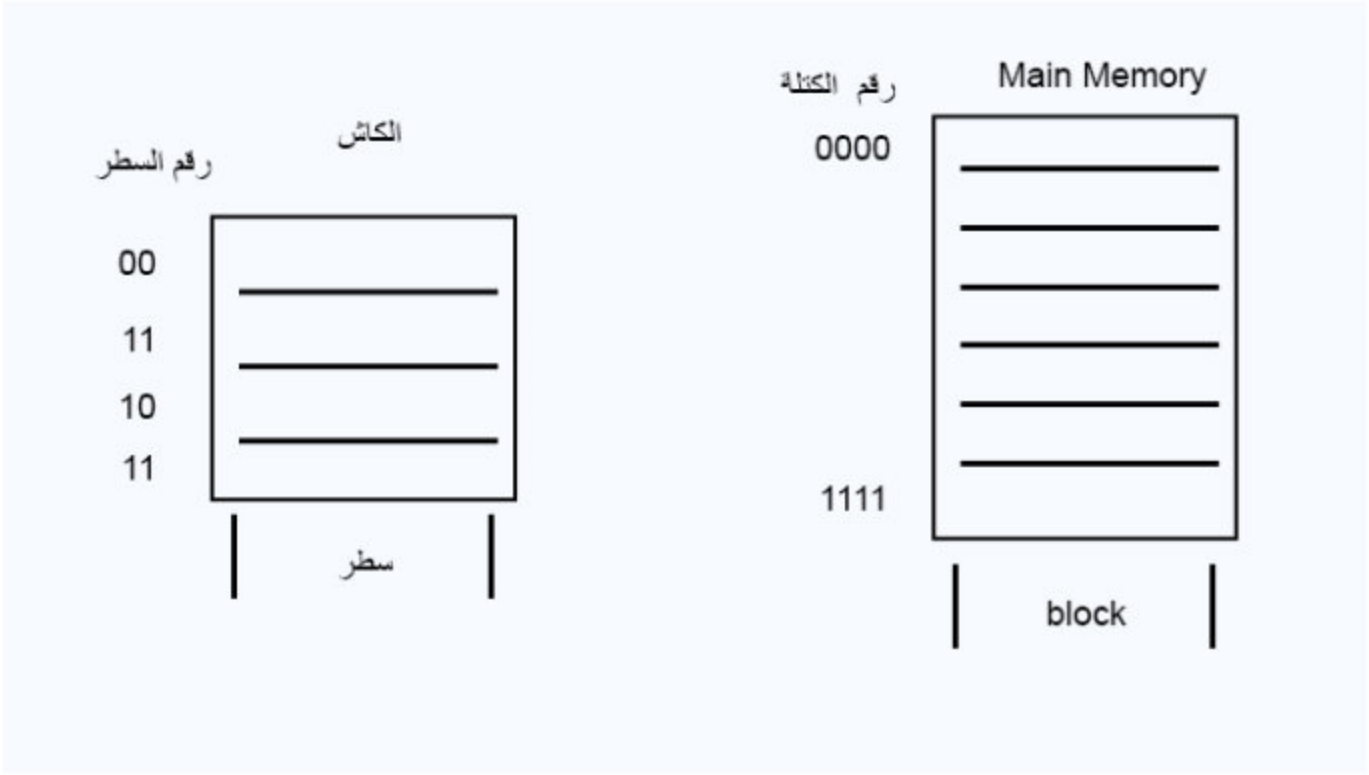
سطر فارغ ==> المعلومات فيه غير مهمة يعني السطر الفارغ ليس شرط أن يكون مرتبط بأصفار

مثال : بفرض ذاكرة رئيسية مولف من 16 كتلة $M = 2^s = 16 \implies s = 4\ bits$

وكاش مؤلف من 4 أسطر $m = 2^r = 4 <==> n = 2\ bits$

لسهولة الشرح نعتبر أن الكتلة تتألف من كلمة واحدة (ليست حالة فعلية أبدا)

$$1^w = 1 ==> w = 0$$



بالعودة للقانون : $i = j\ mod\ m$

0 mod 4 = 0	1 mod 4 = 1	2 mod 4 = 2	3 mod 4 = 3
4 mod 4 = 0	5 mod 4 = 1	6 mod 4 = 2	7 mod 4 = 3
8 mod 4 = 0	9 mod 4 = 1	10 mod 4 = 2	11 mod 4 = 3
12 mod 4 = 0	13 mod 4 = 1	14 mod 4 = 2	15 mod 4 = 3
00 رقم السطر	01	10	11

عدد التاغات هو نفس عدد الكتل المتواجدة في نفس السطر

من البتات الأخرى : بت اسمه v valid عندما يكون السطر فارغ يكون : v = 0

وعندما يحوي كتلة v = 1

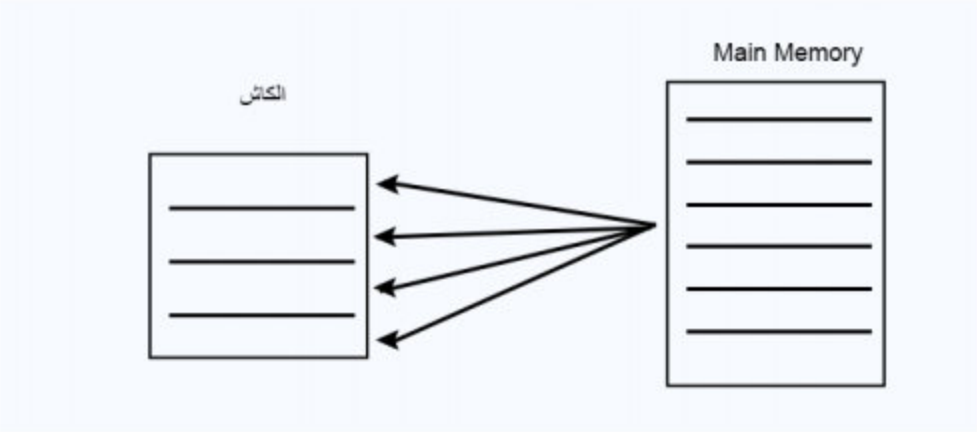
S (4 bits)	
S - r	r

4 - 2 = 2 bits

2 bits

الكتل ذوات الأرقام (رقم 0 و رقم 4 ورقم 8 ورقم 12) ستكون في السطر 0

- التقابل التجميعي : Associative (Full Associative)



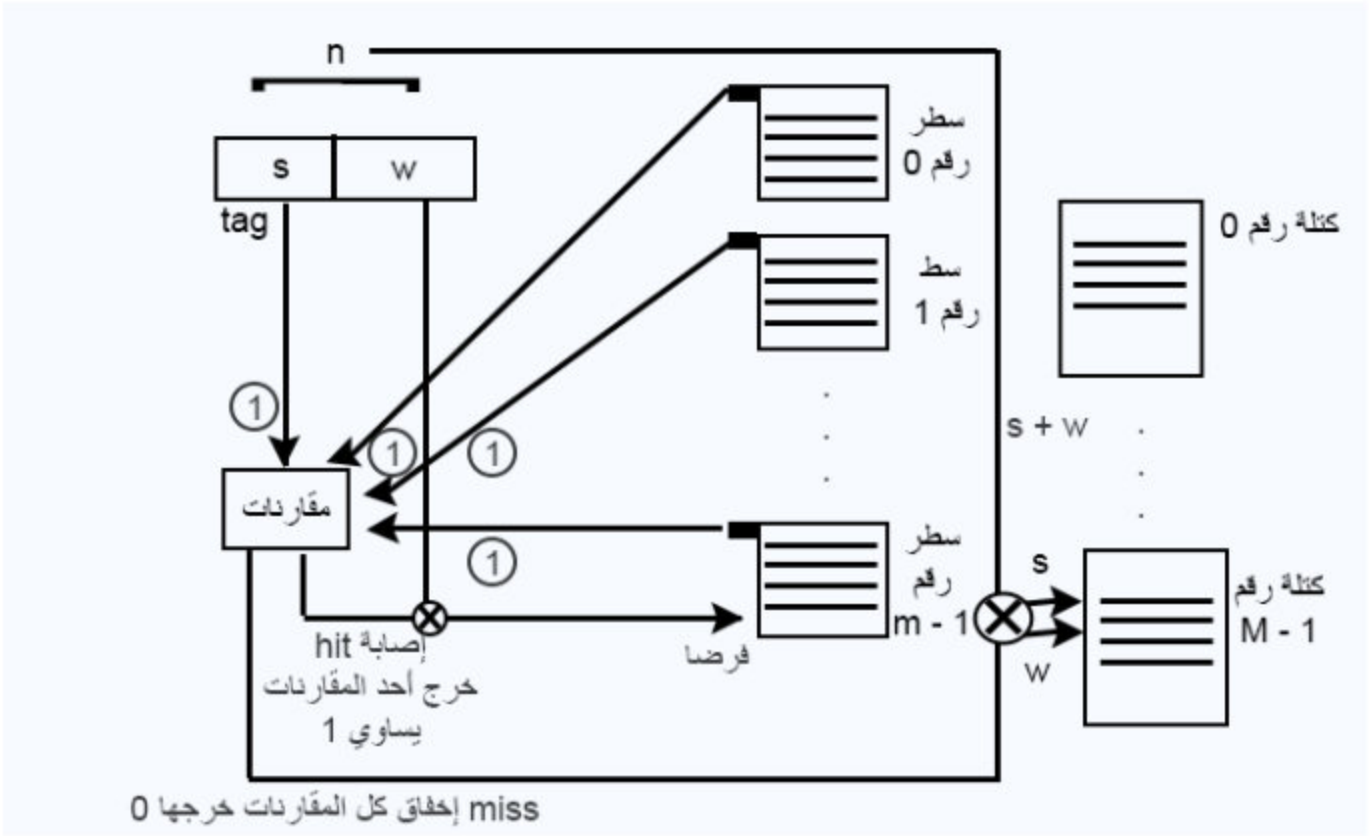
كل كتلة ممكن أن توضع في أي سطر في الكاش

المزايا : ممكن وضع الكتلة في أي سطر (لا يوجد أسطر فارغة في الكاش بل يوجد استبدال)

المساؤئ : عدد المقارنات يساوي عدد أسطر الكاش

أولا : نحتاج مقارنات بعدد الأسطر، ثانيا : إذا كانت إصابة (المقارن الذي خرجه 1 تكون الكتلة موجودة في الكاش أما إذا كانت miss يأخذ w+ s ويذهب إلى الذاكرة الرئيسية

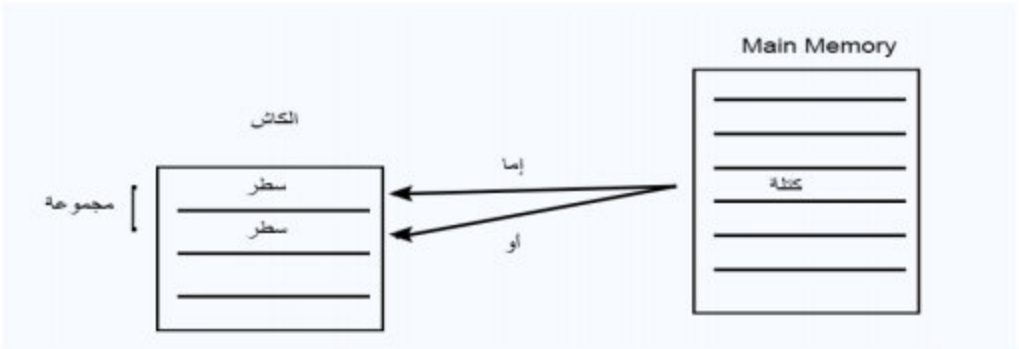
عند الإخفاق يكون جميع المقارنات خرجها : 0

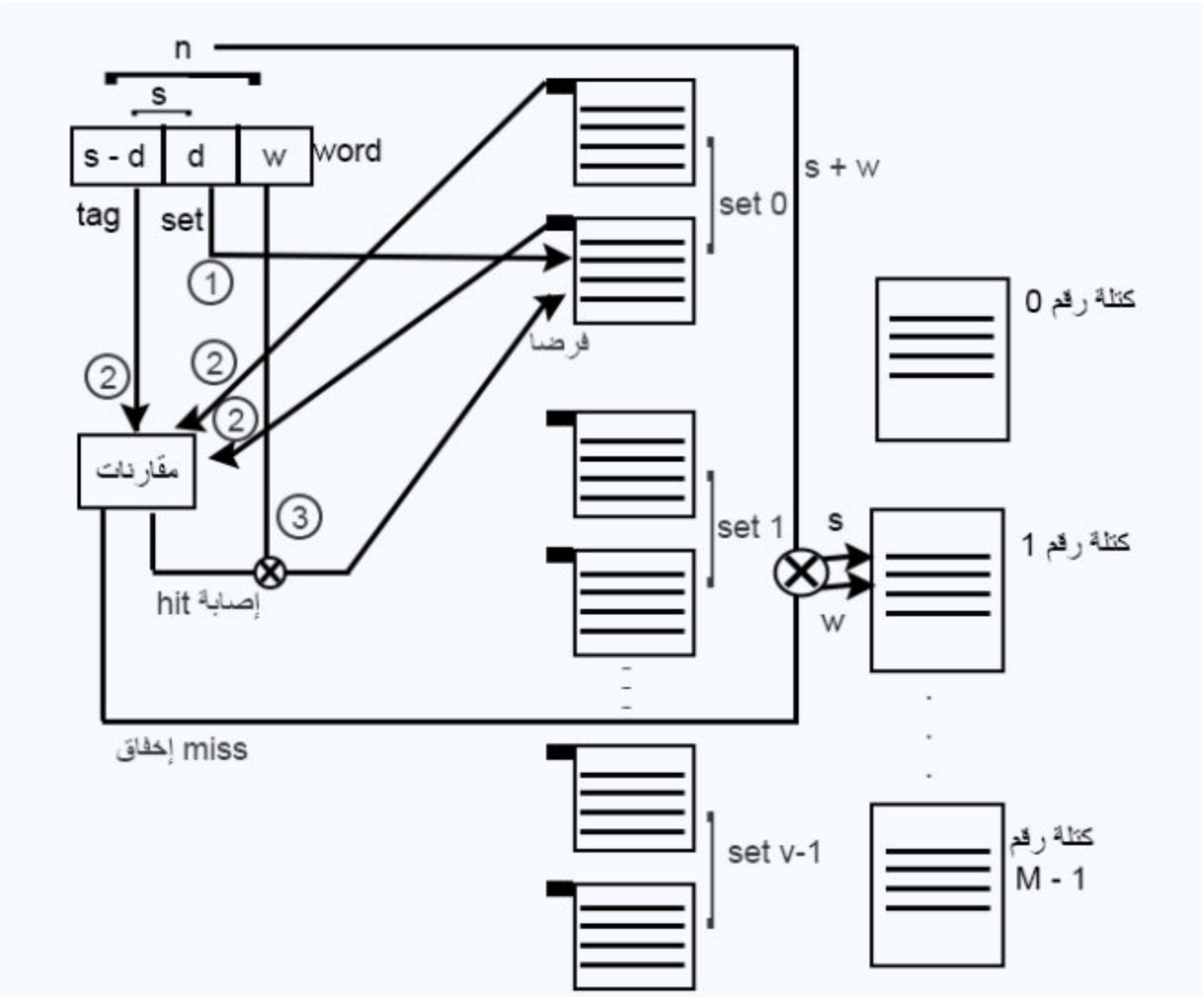


عدد المقارنات = عدد الأسطر

ملاحظة : هنا نحتاج incoder أما في الطريقة الأولى يوجد multiplexer

- التقابل التجميعي في مجموعات ذات k اتجاه k-way set Associative
  طريقة وسط بين الطريقتين السابقتين
  وهي أكثر الطرق المستخدمة في الكاش
  تقسم الكاش إلى مجموعات sets عددها $v = 2^d$ حيث d عدد البتات التي تدل على رقم المجموعة (عنوان) في الكاش وكل مجموعة تتألف من k سطر
  عدد أسطر الكاش $2^r = m = v * k$
  الكتلة توضع في مجموعة محددة حسب العلاقة : $j \bmod v =$ رقم المجموعة
  حيث j رقم الكتلة
  ويمكن وضعها في أي سطر بالمجموعة





عدد المقارنات = عدد أسطر المجموعة

أولا : نذهب إلى المجموعة الأولى فرضا،

ثانيا : نريد معرفة الكتلة هذه بأي سطر موجودة وبالتالي نستخدم مقارنات لل tag (على فرض المقارن ذو الرقم 2 أعطى قيمة ما

ثالثا : في حال الإصابة يكون مكان الكلمة في السطر الذي أعطى المقارن له قيمة 1

رابعا : في حال الإخفاق يذهب إلى الذاكرة الرئيسية ويأخذ s+w ونضعها في الكتلة، يأخذ الكتلة ويضعها في السطر الفارغ في الكاش

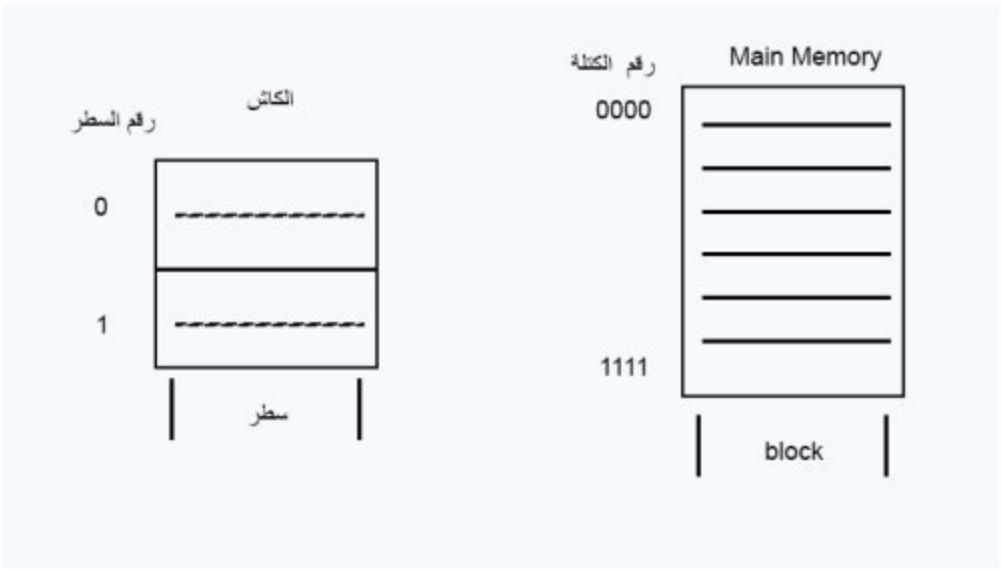
مثال : بفرض ذاكرة رئيسية تحوي 16 كتلة

وبفرض ذاكرة كاش تحوي مجموعتين وكل مجموعة سطرين

$$M = 2^s = 16 ==> s = 4$$
$$V = 2^d = 2 ==> d = 1bit$$

$$k = 2$$

لسهولة الشرح نفرض الكتلة تتألف من كلمة واحدة $2^w = 1 ==> w = 0$



رقم المجموعة 0 mod 2 = 0	1 mod 2 = 1
2 mod 2 = 0	3 mod 2 = 1
4 mod 2 = 0	5 mod 2 = 1
6 mod 2 = 0	7 mod 2 = 1
8 mod 2 = 0	9 mod 2 = 1
10 mod 2 = 0	11 mod 2 = 1
12 mod 2 = 0	13 mod 2 = 1
14 mod 2 = 0	15 mod 2 = 1

s-d : 3 bits و d : 1 bit

الكتل ذوات الأرقام (0 و 2 و 4 و 6 و 8 و 10 و 12 و 14) ستوضع في المجموعة رقم 0

المحاضرة الخامسة

عناصر تصميم الكاش Cache

7 عناصر

- تابع التقابل :
  - أ. المباشر
  - ب. التجميعي
  - ت. تجميعي في مجموعات

  تم شرحهم بالمحاضرة السابقة
- حجم الكاش (سعة الكاش) size of cache

  كلما كانت أصغر كانت أسرع وأرخص
- خوارزمية الاستبدال Replacement Algorithms

عند الحاجة لكلمة وكانت غير موجودة في الكاش (الكتلة التي تحوي الكلمة غير موجودة في الكاش) سيتم جلب الكتلة الخاصة بهذه الكلمة من الذاكرة الرئيسية MM وسيتم وضعها في أحد أسطر الكاش

فإذا كان هذا السطر مشغول سيتم استبدال الكتلة الموجودة فيه بالكتلة المراد جلبها من ال MM. هناك عدة خوارزميات للاستبدال تطبق فقط (في تابع التقابل التجميعي والتجميعي في مجموعات) أما التقابل المباشر لا يوجد له خوارزميات استبدال لأن لكل كتلة سطر خاص فيها.

أ. LRU : least recently used الأقدم استخداما

لكل سطر يوجد بت يسمى use bit (بت الاستخدام) وهو أحد البتات الأخرى الذي يتخزن داخل السطر وعند الوصول لكلمة (أي الوصول للكتلة ثم الكلمة) موجودة ضمن هذا السطر يتم وضع ال use bit الخاص به مساويا للواحد وبقية ال use bit (الخاصة ببقية الأسطر) صفر

(آخر Access نصل إلى كلمة موجودة بكتلة، الكتلة موجودة وسيتم استبدال أحد الكتل الموجودة في أحد هذه الأسطر (عشوائيا) بسطر، السطر لديه use bit هو 1 وبالتالي فهي آخر وحدة استخداما (أجدد وحدة)

كلمة

	u		v	tag			

أ. LFU : least frequently used الأقل تكرارا بالاستخدام

(لكل سطر عدد مرتبط به) وعند الوصول لكلمة موجودة ضمن هذا السطر يتم زيادة هذا العداد (وذلك إذا كانت القيمة الابتدائية صفر) أو نقصان هذا العداد (إذا كانت القيمة الابتدائية واحد) وعند الحاجة سيتم استبدال الكتلة

الموجودة في السطر ذو العداد (الأقل / الأكثر) وإذا كان لدينا عدادات متساوية يكون الاستبدال عشوائي

ت. FIFO الأقدم وجودا

ث. Random عشوائي (مولد رقم عشوائي)

- عدد الكاش : L3 L2 L1

الأبطأ $L3 > L2 > L1$ الأسرع

بالبداية كانت ذاكرة كاش واحدة، ومع التطور بدلا من زيادة حجم الكاش تم زيادة عددها

عند إضافة كاش واحدة يتحسن الأداء بنسبة 80% وعند إضافة كاش ثالث يتحسن الأداء 85%

- هل الكاش موحدة unified أو مشطورة splited

L1 : مشطورة (I_Cache (instruction) , D_Cache (data))

L2,L3: موحدة

- حجم الكتلة : أو السطر (السطر مثل الكتلة لكن فيه معطيات أكثر)

لا يوجد حجم مثالي إنما يتعلق الأداء بالبرنامج المفذ

من الممكن أن ترتفع نسبة الإصابة (كتلة كبيرة)

ومن الممكن أن تسوء نسبة الإصابة (جلب كلمات بعيدة عن الكلمة المطلوبة)

p_cache : L1 : Cori7 32kb

I_cache 32kb

64bit : Cori7

- سياسة الكتابة : write policy

التعديل على أي كلمة بالكاش يجب أن يتعدل على نسختها في الذاكرة الرئيسية (يكفي تعديل كلمة)

متى يتم التعديل على الذاكرة الرئيسية ؟ يوجد طريقتين:

1) Write back الكتابة خلفا أو التعديل عند الاستبدال : يوجد لكل سطر بت اسمه dirty/update bit

عند التعديل على كلمة موجودة في هذا السطر تصبح قيمته = "1"

وعند الاستبدال سيتم نسخ الكتلة المستبدلة إلى الذاكرة الرئيسية (مكان الكتلة المقابلة لها)

إذا هنا يتم التعديل على الكاش وعند الاستبدال يتم على MM

المزايا :

استهلاك طاقة أقل (لأن الذهاب إلى الذاكرة الرئيسية MM يكون قليلا)

المساوئ :

يوجد جزء من الذاكرة الرئيسية غير صالح (معدل في الكاش فقط)

أي ممكن أن يكون هناك معالج أخر طلب نفس الكتلة الموجودة مع المعالج الأول وكل من المعالجين عدل على الكتلة نفسها بشكل مختلف والحل هو جعل جزء من الذاكرة الرئيسية لا يمكن وضعه في الكاش هو المشترك بين المعالجات (uncacheable)

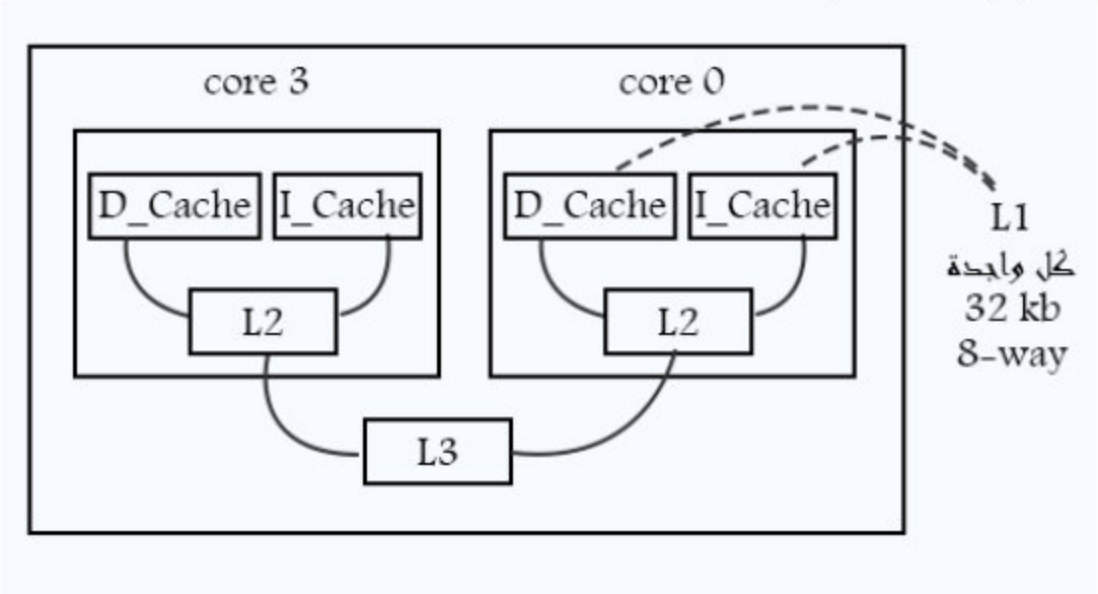
2) الكتابة أثناء النفاذ write through

يتم التعديل على الكاش والذاكرة الرئيسية بنفس الوقت

المساوئ :

يوجد ازدحام على الذاكرة الرئيسية وهنا يوجد حاجة لدارة تراقب مداخل الذاكرة الرئيسية وتعدل على جميع الكاشات الخاصة ببقية المعالجات

نظام ال cache في core i7



حجم الكتلة 64bytes

L1 : كل وحدة 32kb

تستخدم الطريقة الثالثة في التقابل (يقسم إلى مجموعات وكل مجموعة هي 8 أسطر)

L2 : 256kb كل واحدة منهم 8way

L3 : 8MB , 16 way

المعالج 64bits : 1) data bus 64 bits

address bus 36 bits

64-bits cpu : Alu, Reg, Cu

2) أطوال أغلب مسجلاته 64bits

3) تعليماته تتعامل مع معطيات بطول 64bits وأقل

زمن الوصول الوسطي للذاكرة بوجود cache وحدة (الوصول لكاش و MM في نفس الوقب)

$$Ta = h\,Tc + (1-h)Tm$$

Ta : الزمن الكلي الوسطي للوصول للمعلومة effective over all access time

h : معدل الإصابة hit rctlio وتكون قريبة من الواحد وتعطى كنسبة مؤية وتعتمد على عناصر تصميم الكاش

$Ta = Tc <== h = 1$ غير موجودة عندما MM

$Ta = Tm <== h = 0$ غير موجودة عندما $cache$

Tc : cache memory access time one

Tm : main memory access time (100ns)

زمن الوصول الوسطي للذاكرة يوجود 2cache (كاش وMM بنفس الوقت) : (غير مطلوبة)

$$\boldsymbol{Ta = h1Tc1 + (1-h1)h2\ Tc2 + (1-h1)(1-h2)Tm}$$

زمن الوصول الوسطي للذاكرة بوجود 1cache (الوصول لكاش ثم MM)

$$\boldsymbol{Ta = hTc = (1-h)(Tc + Tm)}$$

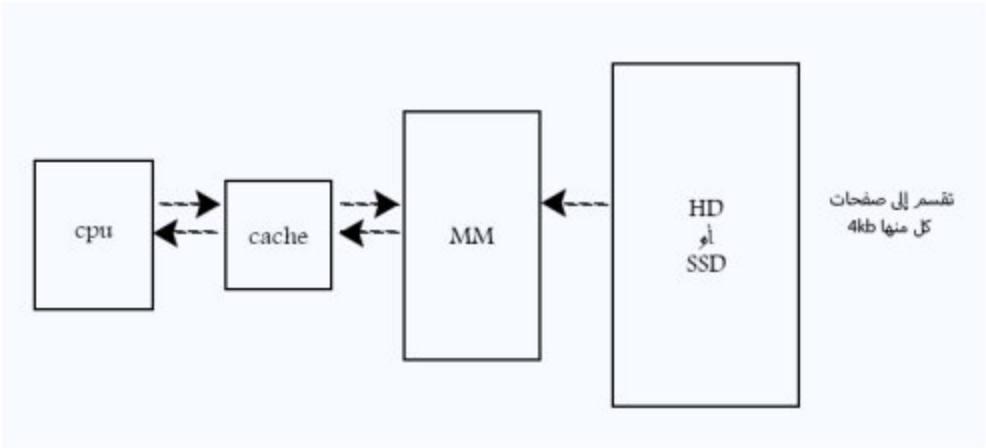
زمن الوصول الوسطي للذاكرة بوجود 2cache (الوصول لكاش ثم MM)

$$Ta = h1Tc1 + (1-h1)h2\,(Tc2 + Tc1) + (1-h1)(1-h2)(Tc1 + Tc2 + Tm)$$

تقنية ال RAID

Redundont array of Independent (Inexpensive) Disk

مجموعة متكررة من الأقراص (السواقات المستقلة)



إن التحسين في أداء (performance) القرص ليس على مستوى التحسين في أداء ال cpu وال memory == > الحل هو RAID

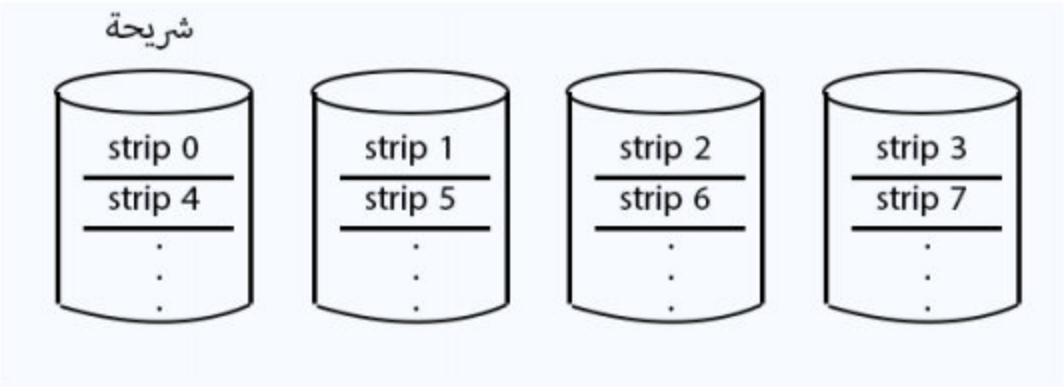
RAID : هو مجموعة من سواقات الأقراص الفيزيائية المستقلة والتي يراها os (operating system) كقرص منطقي واحد (عندما نقول الأقراص نعني بها سواقات الأقراص)

استخدام ال RAID أدى إلى (فوائد ال RAID):

- تحسين الأداء (عن طريق تشريح البيانات data striping على مجموعات من الأقراص يعطي إمكانية القراءة والكتابة على التوازي)
- زيادة السعة
- زيادة الوثوقية Reliability (استرجاع البيانات عند فقدانها) (عند تعطل أحد الأقراص)

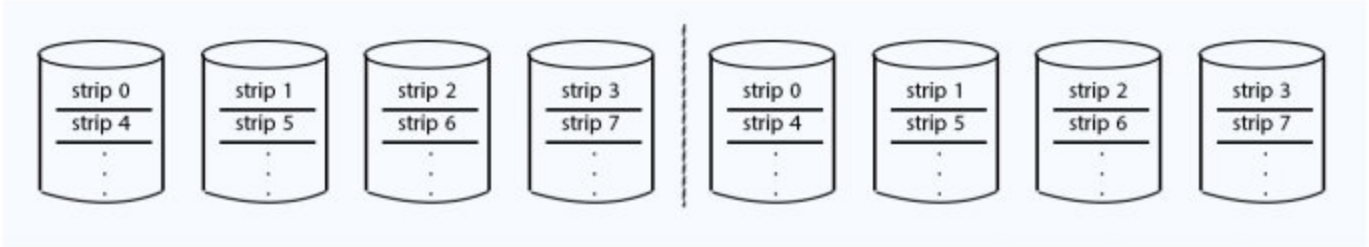
Level of RAID : أثناء الشرح سيتم استخدام 4 أقراص معطيات

- RAID 0



حجم Strip : (16 ---> 128)KB
العدد الأصغري للأقراص : 2
العدد الكلي : N
عدد أقراص المعطيات : N (كل الأقراص هي أقراص معطيات)
يستخدم للبيانات التي تحتاج سرعة في القراءة والكتابة وغير مهمة (فيديو)
لا يصلح أي خطأ

ن RAID 1

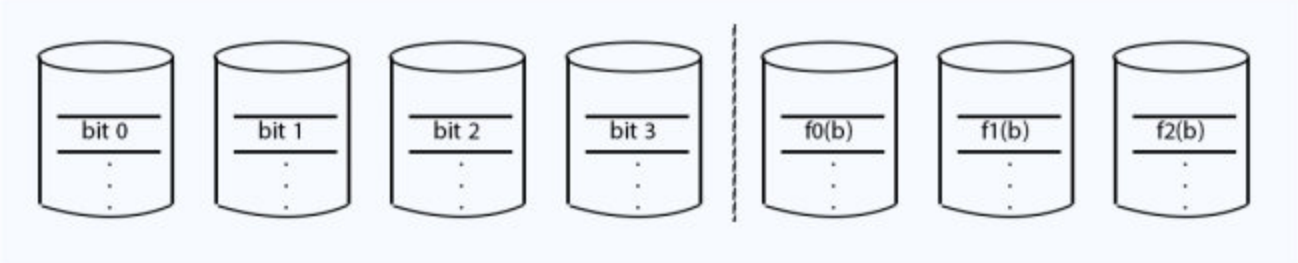


فعليا هو 01
حجم Strip : (16 --> 128)KB
العدد الأصغري للأقراص : 4
العدد الكلي : 2N,3N,...
عدد أقراص المعطيات : N
الاستخدام : للمعلومات المهمة (غالي جدا)
عدد الأخطاء : حسب عدد المجموعة الفائضة (المكررة) وأين يحدث الخطأ

- RAID 2 : تم استخدام أحد خوارزميات كشف وتصحسح الأخطاء (ترميز Hamming) لتصحيح خطأ واحد في N بت يجب إضافة K بت بحيث تتحقق المتراجحة

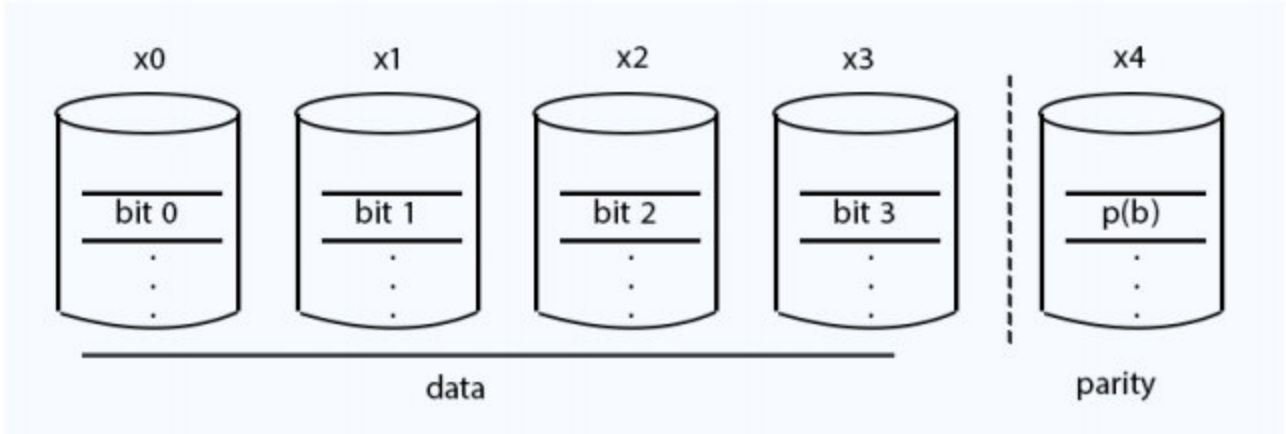
$$2^k - 1 \geq N + k$$

بفرض $N = 4 ==> k = 3$



حجم Strip : 1 bit
العدد الأصغري للأقراص : $2^k - 1 \geq N + k$
العدد الكلي : N + k
عدد أقراص المعطيات : N
الاستخدام : غير مستخدمة / غالية وتحتاج زمن
عدد الأخطاء : يصلح خطأ واحد

1) RAID 3



حجم Strip : 1bit
العدد الأصغري للأقراص : 3
العدد الكلي : N + 1
عدد أقراص المعطيات : N
الاستخدام : تخزين الصور/ التعامل مع الصور
عدد الأخطاء : يصلح خطأ واحد

بفرض حدوث عطل في x1 :

$$x4(i) = x3(i) \oplus x2(i) \oplus x1(i) \oplus x0(i)$$
$$x4(i) \oplus x1(i) \oplus x4(i) = x3(i) \oplus x2(i) \oplus x1(i) \oplus x0(i) \oplus x4(i) \oplus x1(i)$$
$$x1(i) = x4(i) \oplus x3(i) \oplus x2(i) \oplus x0(i)$$

ملاحظة :
⊕ تعني العملية xor

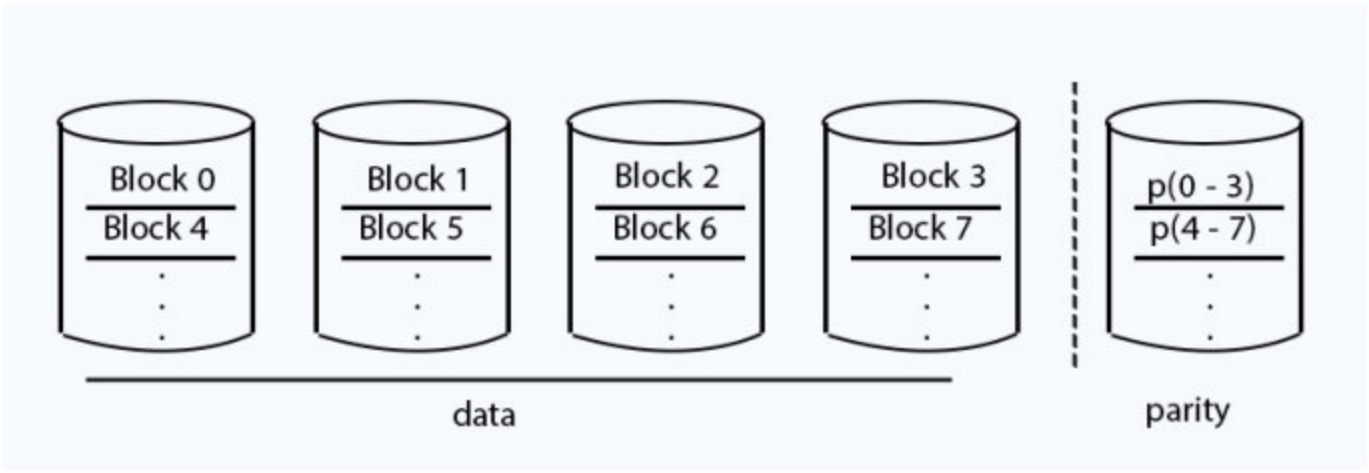
تذكرة بعملية xor : x,y مداخل

x	y	out
0	0	0
0	1	1
1	0	1
1	1	0

بفرض تعديل x3 :

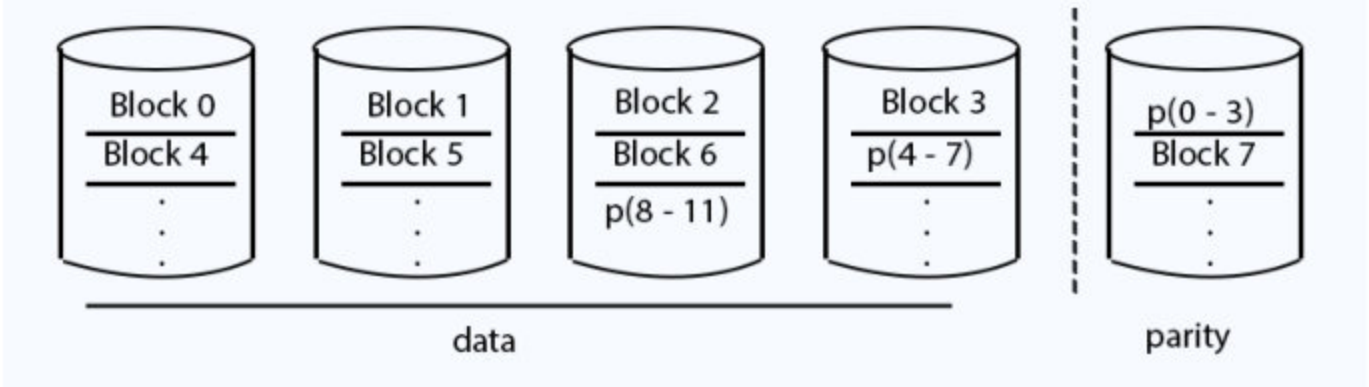
$$x'4(i) = x'3(i) \oplus x2(i) + x1(i) + x0(i)$$
$$x'3(i) = x'4(i) \oplus x2(i) + x1(i) + x0(i)$$

RAID 4



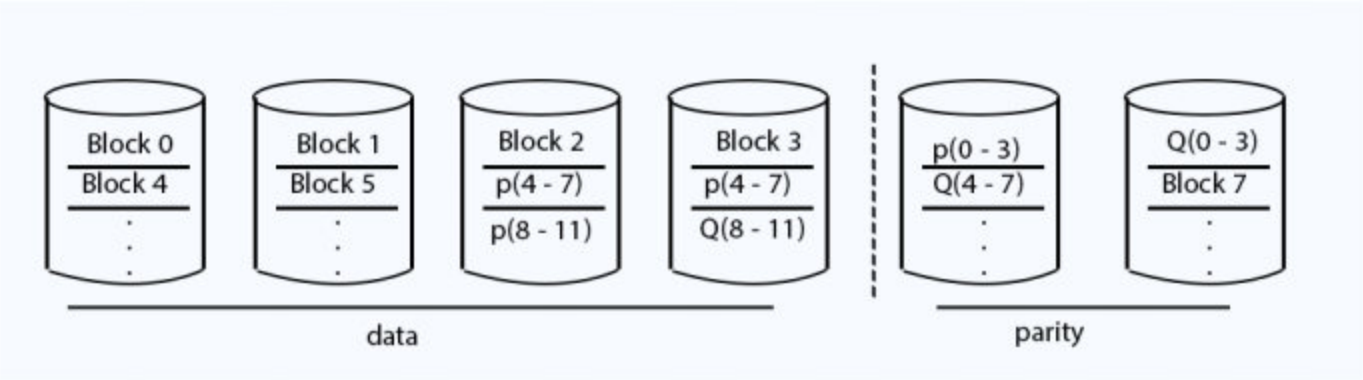
حجم block : (16 –-> 128)KB
العدد الأصغري للأقراص : 3
العدد الكلي : N + 1
عدد أقراص المعطيات : N
الاستخدام : لم يتم استخدامه لوجود ازدحام على parit disk
عدد الأخطاء : يصلح خطأ واحد

RAID 5



حجم block : (16 --> 128)KB
العدد الأصغري للأقراص : 3
العدد الكلي : N + 1
عدد أقراص المعطيات : N
الاستخدام : data server , web server
عدد الأخطاء : يصلح خطأ واحد

ن RAID 6



حجم block : (16 --> 128)KB
العدد الأصغري للأقراص : 4
العدد الكلي : N + 2
عدد أقراص المعطيات : N
الاستخدام : data server , web server
عدد الأخطاء : يصلح خطأين

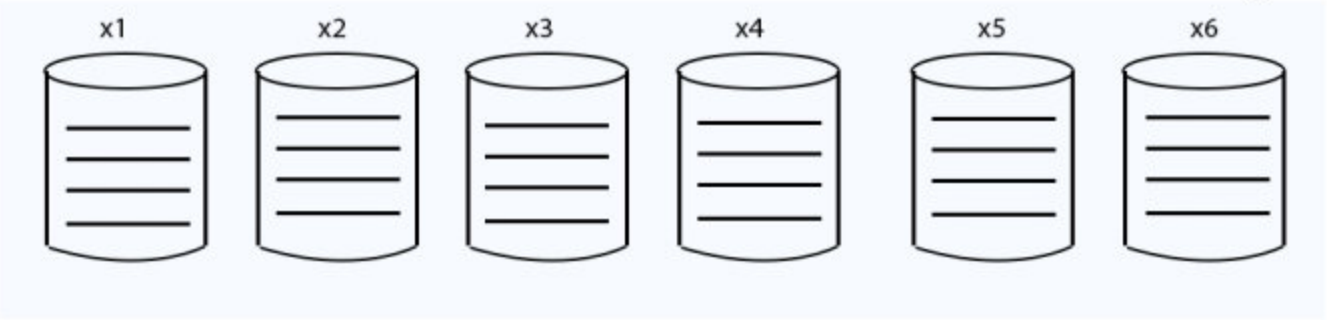
سؤال دورة :

لدينا نظام RAID يتألف من 6 أقراص "سواقات" وكل قرص سعته 100GB

أ. حدد سعة التخزين المتوفرة للمعطيات من أجل مستويات أنظمة 6 , 5 , 4 , 3 , 2 , 16 , 0

ب. بفرض حدث عطل في القرص الثالث. بين كيف يتمكن نظام RAID من استعادة المعطيات الموجودة على هذا القرص

الحل : أ.



1) للنظام RAID 0 = level 0

$$6 * 100G = 600\ GB$$

N = 6 قرص

2) للنظام RAID 1 = level 1

2N = 6

N = 3

$$3 * 100\ GB = 300\ GB$$

3) للنظام RAID 3 = level 3

N + 1 = 6

N = 5

$$5 * 100\ GB = 500\ GB$$

4) للنظام RAID 4 = level 4

N + 1 = 6

N = 5

$$5 * 100\ GB = 500\ GB$$

5) للنظام RAID 5 = level 5

N + 1 = 6

N = 5

$$5 * 100\,GB = 500GB$$

6) للنظام level 6 = RAID

N + 2 = 6

N = 4

$$4 * 100\,GB = 400GB$$

7) للنظام level 2 = RAID 2

$$2^k - 1 \geq N + k \;\; ==> k = 3$$
$$N = 6 - k$$
$$N = 3$$
$$3 * 100GB = 300GB$$

ب.

$$x6 = x1 \oplus x2 \oplus x3 \oplus x4 \oplus x5$$

X6 : القرص الذي تم تخزين معلومات الاستعادة فيه panity

نجمع $x3 \oplus x6$ إلى الطرفين

$$x6 \oplus x3 \oplus x6 = x1 \oplus x2 \oplus x3 \oplus x4 \oplus x5 \oplus x3 \oplus x6$$

$x6 \oplus x6 = 0$ وكذلك $x3 \oplus x3 = 0$ بالاختصار

$$x3 = x1 \oplus x2 \oplus x4 \oplus x5 \oplus x6$$

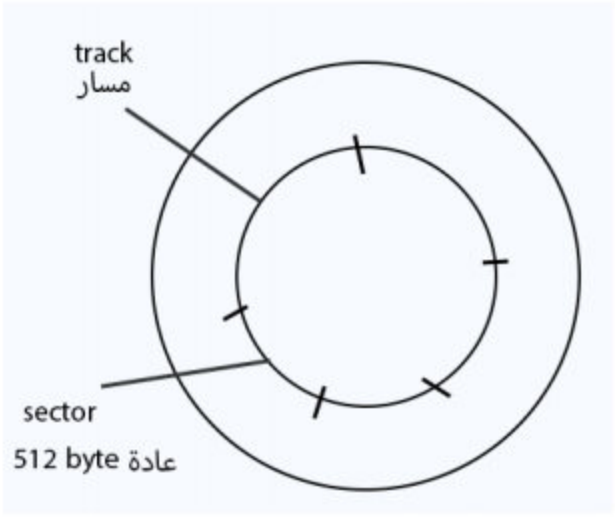
زمن الوصول ل HD

Access time for HD

Total Average Access time

Total Average transfer time

$$Ta = Ts + \frac{1}{2r} + \frac{b}{Nr}$$



Ts : seek time زمن البحث الوسطي : الزمن اللازم للوصول إلى المسار track

$\frac{1}{2r}$: rotational delay زمن التأخير الدوراني الوسطي : الزمن اللازم لوضع رأس القراءة / الكتابة على القطاع sector

$x = \frac{1}{r}$ الزمن اللازم لإتمام الدورة

r : rotation speed سرعة الدوران RPS : Round Per Second

N : Number of bytes on track عدد البايتات على المسار

b : Number of bytes to be transfer عدد البايتات المراد نقلها $x = \frac{b}{Nr}$

توزيع البيانات الخاصة بالملف

- القطاعات موجودة في مسارات متجاورة (أو متوازية على كل الأقراص (الأسطوانة (cylinder))

(sequential organization) التنظيم التتابعي أو التسلسلي

- القطاعات موجودة في كل الأقراص

(random organization) التنظيم العشوائي

مسألة :

لدينا قرص له زمن بحث وسطي 4 msec وسرعة الدوران له 15000 rpm دورة في الدقيقة، عدد القطاعات في المسلك هو 500 قطاع والمطلوب:

حدد الزمن الكلي اللازم لعملية قراءة ملف يتألف من 2500 قطاع في الحالتين :

1) Sequential organization التنظيم التتابعي : الملف يخزن في قطاعات موجودة في مسالك متجاورة
2) Randoum organization التنظيم العشوائي : الملف يخزن في قطاعات موزعة عشوائيا على القرص

الحل :

1) التنظيم التتابعي / التسلسلي

عدد المسالك (track) هو $\frac{\text{عدد قطاعات الملف}}{\text{عدد قطاعات المسلك}} = \frac{2500}{500} = 5$

ولدينا $r = 15000\ rpm$ سرعة الدوران

rpm : تعني دورة بالدقيقة

زمن البحث الوسطي Ts = 4msec

المطلوب : الزمن الكلي أي Ta

الزمن اللازم لقراءة المسلك الأول 4msec (زمن البحث)

التأخير الدوراني :

$$\frac{1}{2r} = \frac{1}{2 * \frac{15000}{60}} = 2\ msec$$ مسلك واحد

$$\frac{b}{rN} = \frac{500 * 512}{\frac{15000}{60} * (500 * 512)} = 4msec$$ عدد قطاعات المسلك حجمها بالبايت

حيث: b : عدد قطاعات المسلك * حجمها بالبايت

N : عدد البايتات على المسلك

لدينا 500 قطاع وكل قطاع 512 بايت لذلك ضربنا 500 * 512

سرعة الدوران للقرص ب sec : $\frac{15000}{60}$

الزمن اللازم لقراءة أول مسلك $4 + 4 + 2 = 10\ msec$

الزمن اللازم لقراءة بقية المسالك (يتم إهمال زمن البحث لأن المسالك متجاورة (تنظيم تسلسلي))

$4 * (2 + 4) = 24\,msec$ بقية المسالك

الزمن الكلي لقراءة الملف

$$24 + 10 = 34msec = 0.034sec$$

القاعدة : نحسب زمن مسلك واحد + يضاف إليها زمن أربع مسالك متبقية مع إهمال زمن البحث Ts =0

2) التنظيم العشوائي :

$$\text{الزمن الازم لقراءة قطاع واحد} = Seek\ Time + \frac{1}{2r} + \frac{b}{rN}$$

نحسب لقطاع واحد ثم نضرب بعدد القطاعات

$$seek\ time = 4\,msec \quad ---1$$

$$\frac{1}{2r} = \frac{1}{2 * \frac{15000}{60}} = 2\,msec \quad ---2$$

حجم قطاع واحد

$$\frac{b}{rN} = \frac{512}{\frac{15000}{60} * (500 * 512)} = 0.008\,msec \quad ---3$$

$$\frac{\text{الزمن اللازم}}{\text{لقراءة قطاع واحد}} = 4 + 2 + 0.008 = 6.008\,msec$$

الزمن الكلي : 2500 * 6.008 = 15.02 sec

حيث 6.008 : زمن قطاع واحد

2500 عدد قطاعات الملف

انتهت مقررالدكتور مازن...

نسأل الله التوفيق لنا ولكم ....

اكتب ملاحظات: